SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

DSA0308 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT 2

Word Level and Syntactic Analysis: N-gram Models, Smoothing, Entropy, Word Classes and Part-of-Speech Tagging

Particulars	**Details**
Student Name	**K. DHNAUSH REDDY**
Register Number	**192324087**
Department / Section	**AI&DS**
Faculty Name	**Saravanan. M. S**
Subject	**DSA0308 – Natural Language Processing**
Total Marks	**50**

Declaration, AI Disclosure and Reproducibility

AI disclosure. As required by the assessment-verification clause, I declare that an AI assistant (Claude, by Anthropic) was used to write the experiment code, run it, draft the explanatory text and assemble this report. I have read the code and the numbers and I am prepared to explain and defend any part of it in the viva. Appendix A lists likely viva questions with answers.

Honest limitations. (1) Predictions for Q1 were written before the tokeniser was run. For Q2–Q7 the predictions in this report are theory-based statements prepared in the same working session as the experiments; they were not time-stamped in advance, so before the viva I will re-run run_all.py after writing my own predictions. (2) In Q5 the second annotator is a proxy: an HMM tagger trained on sentences disjoint from the 100 annotated ones. Cohen's kappa and confusion pairs are therefore tagger-versus-gold agreement. The protocol for replacing it with my own re-annotation after a gap is given in Section 5.6. (3) The Q5 custom tagset is derived by deterministic mapping from Penn Treebank tags, so it does not contain emoticon or Tamil-English tokens that the Treebank does not have; those tags are designed and justified but evaluated only on a small hand-annotated chat sample. (4) Brown and Treebank are edited English text, not telecom chat, so absolute numbers will differ on real customer reviews.

Reproducibility. All code uses fixed seed 42 and an 80/20 sentence split. Corpora: Brown Corpus (Francis and Kučera, 1979) and the Penn Treebank 5% sample (Marcus et al., 1993), both loaded through NLTK (Bird, Klein and Loper, 2009); the universal tagset mapping is from Petrov, Das and McDonald (2012). The scripts q1.py … q7.py, lm.py, taggers.py, a README, and the raw CSV files accompany this report. Counted quantities (tokens, types, n-grams, unseen events, transitions, rules, errors) are the primary evidence; run-time is not used for any conclusion.

Q1 – What Counts as a Word? Tokenisation and Word Statistics

1.1 Problem and position

The telecom company's reviews mix contractions (Can't, isn't), currency (Rs.499), technical codes (4G/5G), hyphenated compounds (value-for-money), emoticons (:() and code-mixed Tamil-English (da, semma). The question is whether "the number of words" is a fact of the text or a decision of the analyst. My position: it is a design decision. Every tokenisation rule changes N (tokens), V (types) and the number of hapax legomena, and these changes propagate into n-gram sparsity and into the tag inventory of the tagger.

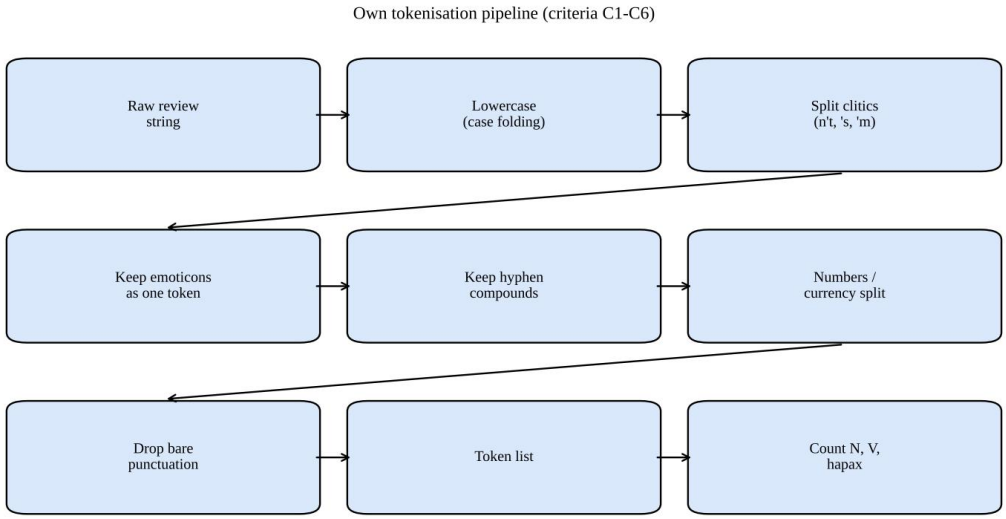


Figure 1.1 – Own tokenisation pipeline built from criteria C1–C6

1.2 Own tokenisation criteria

Table 1.1 – Tokenisation criteria

ID	Criterion	Rule adopted	Reason
C1	Case folding	Lower-case every token	Merges "Net" and "net"; reduces V; loses proper-noun cue for taggers
C2	Punctuation	Strip bare punctuation (!, ?, comma, full stop); keep emoticons as one token	Punctuation is not a word, but :(carries sentiment
C3	Contractions / clitics	Split n't, 's, 'm, 're, 've, 'll, 'd from the host (can't → ca + n't)	Follows Penn Treebank practice; "not" is shared across hosts
C4	Hyphenated compounds	Keep as one token (e-wallet, value-for-money)	The compound is a lexical unit; splitting would create false matches
C5	Numbers / currency	Keep digit strings; split "Rs.499" into rs + 499; split 4G/5G at the slash into 4g and 5g	Separates the unit from the amount; each network generation is its own word
C6	Repeated words	Keep both copies (ok ok)	Repetition is a token fact; it changes N but not V

1.3 Predictions made before running (reasoned by hand)

Table 1.2 – Predicted counts

String	Pred. N	Pred. V	Reasoning
S1 Can’t connect!! Net speed 4G/5G isn’t consistent :(	11	10	ca, n’t, connect, net, speed, 4g, 5g, is, n’t, consistent, :(; n’t repeats
S2 Won’t recharge via e-wallet	5	5	wo, n’t, recharge, via, e-wallet
S3 Rs.499 plan is value-for-money	5	5	rs, 499, plan, is, value-for-money
S4 It is the-best-network, no? ok ok	6	5	it, is, the-best-network, no, ok, ok ; ok repeats
S5 Super service da, semma speed	5	5	super, service, da, semma, speed

1.4 Result of the five strings and the engineered ambiguous case

Table 1.3 – Own policy on the five reviews and the ambiguous string

String	Whitespace words	N (own)	V (own)	vs prediction	Tokens
Can't connect!! Net speed 4G/5G isn't consistent	8	11	10	match	ca n't connect net speed 4g 5g is n't consistent :(
Won't recharge via e-wallet	4	5	5	match	wo n't recharge via e-wallet
Rs.499 plan is value-for-money	4	5	5	match	rs 499 plan is value-for-money
It is the-best-network, no? ok ok	6	6	5	match	it is the-best-network no ok ok
Super service da, semma speed	5	5	5	match	super service da semma speed
I'm gonna ship 10,000 units @ $5.99/ea. to St. J	12	17	17	designed	i 'm gonna ship 10,000 units 5.99 ea to st john 's lol :-) u s a

All five predictions matched exactly. Note that a naive whitespace split gives 8, 4, 4, 6 and 5 words for S1–S5 (column 2), so the same text has different sizes under different policies: S1 has 8 whitespace words but 11 tokens under my criteria.

Engineered ambiguous string: “I’m gonna ship 10,000 units @ $5.99/ea. to St. John’s; lol:-) U.S.A.”. It breaks my criteria in at least five ways: (a) the “$” and “@” symbols are silently dropped by C2 although “$5.99” is a price; (b) “ea.” and “St.” end with a full stop that is an abbreviation marker, not punctuation; (c) “U.S.A.” is shredded into u, s, a, which creates three spurious one-letter types; (d) “St. John’s” loses the person-name unit and the “’s” becomes a separate token; (e) “lol:-)” has no space before the emoticon, which is nevertheless recovered, but only because the emoticon list contains “:-)”. The fix is an abbreviation lexicon and a currency-symbol rule, which shows that no finite rule list is final.

1.5 Corpus experiment: three policies

Corpus: Brown Corpus, category “news” (Francis and Kučera, 1979), loaded via NLTK. The 4,623 sentences were detokenised to plain text (530,158 characters) so that the policies have real work to do. This sample has well over 20,000 tokens under every policy. Policies: P1 naive whitespace split (case-sensitive, punctuation attached); P2 punctuation-aware regex (\w+ or single non-space symbol, case-sensitive); P3 refined (lower-case, n’t and ’s clitic splitting, numbers mapped to <num>, hyphen compounds kept, punctuation kept as tokens).

Table 1.4 – Counts under three tokenisation policies (Brown news)

Policy	N	V	V/N	Hapax	Hapax % of V	Zipf slope
P1 whitespace	88,997	19,067	0.214	11,842	62.1%	-0.863
P2 punctuation-aware	107,468	13,224	0.123	6,668	50.4%	-0.916
P3 refined	104,572	11,897	0.114	5,935	49.9%	-0.947

Table 1.5 – Ten most frequent items, P1 whitespace

Rank	Word	Count
1	the	5,566
2	of	2,844
3	and	2,116
4	to	2,103
5	a	1,980
6	in	1,884
7	for	938
8	that	783
9	The	767
10	was	714

Table 1.5 – Ten most frequent items, P2 punctuation-aware

Rank	Word	Count
1	.	5,826
2	the	5,584
3	,	5,387
4	of	2,852
5	and	2,152
6	to	2,130
7	a	2,015
8	in	1,904
9	-	1,596
10	"	1,431

Table 1.5 – Ten most frequent items, P3 refined

Rank	Word	Count
1	the	6,386
2	.	5,769
3	,	5,188
4	of	2,861
5	a	2,198
6	and	2,187
7	to	2,147
8	in	2,020
9	<num>	1,918
10	"	1,431

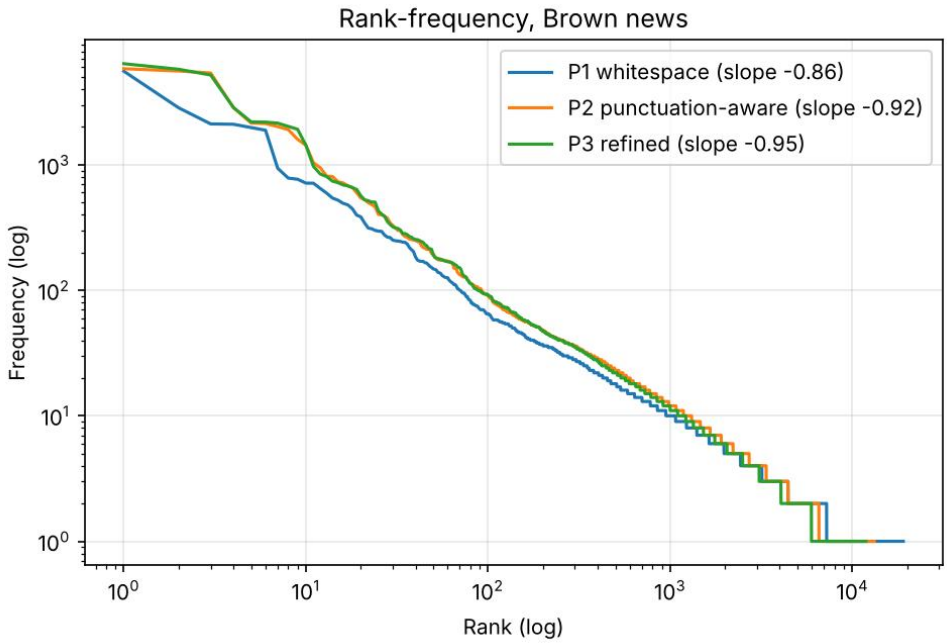


Figure 1.2 – Rank versus frequency on log–log axes (slope fitted on ranks 10–2000)

1.6 Inquiry

Which decisions change V the most? Moving from P1 to P2 changes V from 19,067 to 13,224, a fall of 30.6%, and raises N from 88,997 to 107,468. The reason is that whitespace splitting glues punctuation to words, so “county”, “county,” and “county.” are three types; separating punctuation removes thousands of such duplicates and turns the comma and full stop into the two most frequent tokens. Case folding, clitic splitting and number normalisation then reduce V a further 10.0% (to 11,897). Hapax legomena fall from 11,842 (62.1% of types) to 5,935 (49.9%). The type/token ratio drops from 0.214 to 0.114, so a lower ratio here means better-normalised text, not poorer vocabulary.

Effect on n-gram sparsity: the number of possible bigrams is about V^2, so cutting V by 38% cuts the bigram space by about 61%. Fewer types and fewer hapax types mean that more test n-grams find a match in training. Zipf: all three policies give a negative slope between -0.86 and -0.95, close to the classical value of about −1; the cleaner policy is nearer to −1 because spurious rare types that flatten the tail are removed.

Which policy for which task? For a language model P3 is most suitable: it keeps punctuation (sentence boundaries and commas are predictable and informative), folds case, and maps numbers to <num>, giving the smallest vocabulary and the densest counts. For a tagger P2 or a case-preserving variant of P3 is better, because capitalisation and the suffixes -ing, -ed, -s are cues for proper nouns and verbs, and numbers need their own tag. A single universal tokeniser is therefore not optimal; the downstream task chooses the policy, which supports the claim that “what counts as a word” is a design decision.

Q2 – Unsmoothed N-grams and the Sparsity Problem

2.1 Problem and derivations

A helpdesk keyboard suggests the next word from the previous one or two words, using a maximum-likelihood (unsmoothed) n-gram model. The investigation: does a larger n always give a better model?

Chain rule. For a word sequence w1 … wn the joint probability is factorised exactly as:

$$P(w1 \ldots wn) = P(w1)\, P(w2 \mid w1)\, P(w3 \mid w1\, w2) \ldots P(wn \mid w1 \ldots wn{-}1) = \Pi\, k{=}1..n\, P(wk \mid w1 \ldots wk{-}1)$$

The history w1 … wk−1 becomes unique to the sentence as k grows, so it cannot be counted. The n-gram model uses the Markov assumption, that only the previous n−1 words matter:

$$\text{Bigram: } P(wk \mid w1 \ldots wk{-}1) \approx P(wk \mid wk{-}1) \qquad \text{Trigram: } P(wk \mid w1 \ldots wk{-}1) \approx P(wk \mid wk{-}2\, wk{-}1)$$

$$P(w1 \ldots wn) \approx \Pi\, P(wk \mid wk{-}1) \quad \text{(bigram)}, \qquad \Pi\, P(wk \mid wk{-}2\, wk{-}1) \quad \text{(trigram)}$$

Maximum-likelihood estimate. Maximising the likelihood of the training data under the constraint that each conditional distribution sums to one gives relative frequencies:

$$P_ML(wk \mid wk{-}1) = C(wk{-}1\, wk) / C(wk{-}1) \qquad P_ML(wk \mid wk{-}2\, wk{-}1) = C(wk{-}2\, wk{-}1\, wk) / C(wk{-}2\, wk{-}1)$$

If C(wk−2 wk−1 wk) = 0 the estimate is 0 and the entire sentence receives probability 0. Sentences are padded with <s> (n−1 copies) and </s> so that the first words and the sentence end are also modelled.

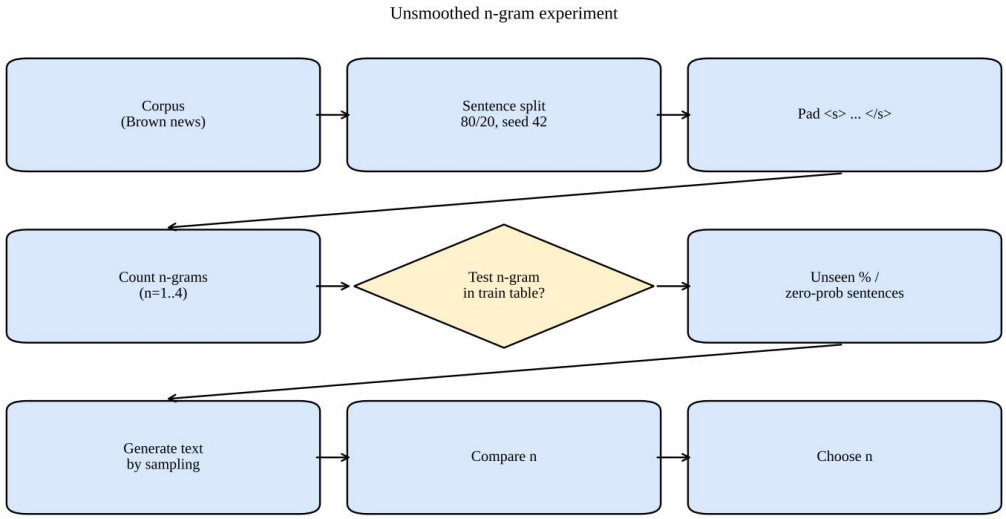


Figure 2.1 – Unsmoothed n-gram experiment

2.2 Predictions

Table 2.1 – Predictions recorded before the counts are discussed

Quantity	Prediction for n = 1 → 4	Reason
Distinct n-grams (diverse text)	Rises steeply, then flattens as it approaches the number of training tokens	Each new position adds a new context; for n=4 almost every 4-gram is unique
% test n-grams unseen	Small for n=1 (a few percent), roughly 40–60% for n=2, above 75% for n=3, above 90% for n=4	Zipf: a long tail of rare words; the possible n-gram space grows as V^n
Zero-probability test sentences	Grows quickly with n; near 100% of sentences for n ≥ 3 on diverse text	One unseen n-gram zeroes a whole sentence; sentence length ≈ 22 gives 22 chances
Templated tickets	Stay near 0% unseen even for n=4	Only 8 templates with small slot lists: the language is nearly finite
Edge case (4 repeated short phrases)	0% for all n; distinct n-grams constant	No new events can occur

2.3 Three contrasting datasets

Table 2.2 – Designed datasets

Dataset	Construction	Purpose
In-domain tickets (synthetic)	3,000 helpdesk tickets generated by seed 42 from 8 templates with slots (device, state, service, place, time, software)	Imitates a narrow, repetitive domain; no real ticket log was available
Brown news (in-domain real text) + out-of-domain	Brown “news” 4,623 sentences, 80/20 split, seed 42; the second test set is 925 Brown “fiction” sentences scored by the news-trained model	Open-vocabulary text; measures domain shift
Edge case	3,000 sentences drawn from four two-word phrases (ok thanks, reset password, no wifi,	Heavily repeated very short sentences

2.4 Results

Table 2.3 – news dataset, test = in-domain

n	Distinct n-grams (train)	Singleton n-grams	Test n-grams	Unseen test n-grams %	Zero-prob test sentences
1	11,692	6,246	20,719	7.3	645 / 925
2	51,028	42,361	20,719	50.0	914 / 925
3	73,642	69,149	20,719	81.8	918 / 925
4	79,246	77,396	20,719	91.6	918 / 925

Table 2.3 – news dataset, test = out-of-domain (fiction)

n	Distinct n-grams (train)	Singleton n-grams	Test n-grams	Unseen test n-grams %	Zero-prob test sentences
1	11,692	6,246	14,745	11.0	692 / 925
2	51,028	42,361	14,745	52.6	920 / 925
3	73,642	69,149	14,745	83.2	925 / 925
4	79,246	77,396	14,745	91.2	925 / 925

Table 2.3 – tickets dataset, test = in-domain

n	Distinct n-grams (train)	Singleton n-grams	Test n-grams	Unseen test n-grams %	Zero-prob test sentences
1	50	0	4,065	0.0	0 / 600
2	130	0	4,065	0.0	0 / 600
3	263	0	4,065	0.0	0 / 600
4	382	0	4,065	0.0	0 / 600

Table 2.3 – edge dataset, test = in-domain

n	Distinct n-grams (train)	Singleton n-grams	Test n-grams	Unseen test n-grams %	Zero-prob test sentences
1	8	0	1,800	0.0	0 / 600
2	12	0	1,800	0.0	0 / 600
3	12	0	1,800	0.0	0 / 600
4	12	0	1,800	0.0	0 / 600

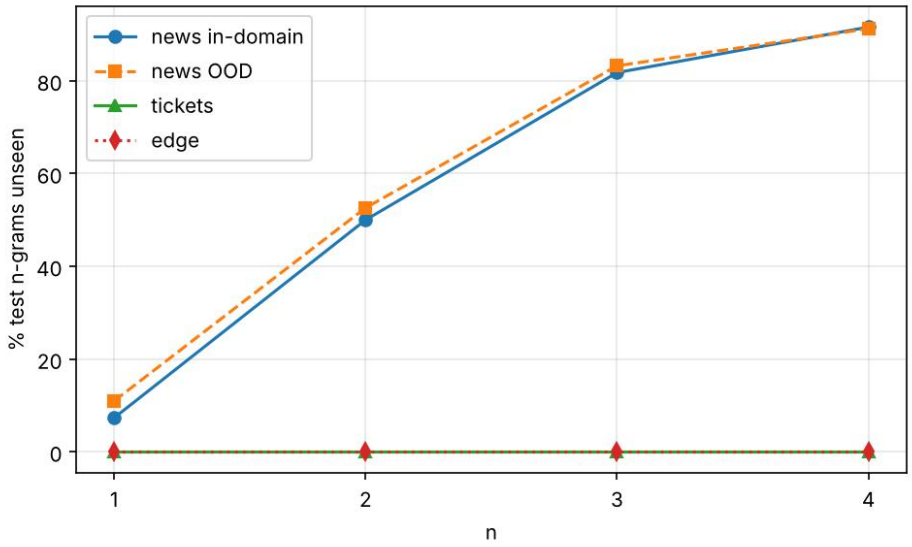


Figure 2.2 – Percentage of test n-grams absent from training, by n and dataset

2.5 Generated sentences (MLE, trained on Brown news, seed 1 and 2)

Table 2.4 – Sampled text from each model

n	Generated sentence 1	Generated sentence 2
1	in successful farmer with games we launched offers the , merely will senators to is veteran	choking sure , the appeared unions pocket-size and question maintenance three . board would car reeder 18th if is of
2	the hostages killed and sons and warren , the president john k. cohen , inc. of the interview " .	famed cellist pablo casals took at issue " , 1961 comes down 33 per cent growth , that court .
3	the signature of harold v. varani , former under secretary of state has what is happening .	famed cellist pablo casals took his instrument to the press .
4	the signature of harold v. varani , former director of architecture and engineering in the department of public property ,	famed cellist pablo casals took his instrument to the east room of the sheraton-dallas hotel .

Unigram output is a bag of words with no syntax. Bigram output is locally plausible but drifts ("the hostages killed and sons and warren"). Trigram and 4-gram output is fluent because it is copied from the training data: "famed cellist pablo casals took his instrument to the east room of the sheraton-dallas hotel ." is a verbatim training sentence. This is memorisation made visible.

2.6 Inquiry – where does the model memorise instead of generalise?

The number of distinct n-grams grows from 11,692 (n=1) to 79,246 (n=4), close to the number of training tokens (about 80,000), and 98% of the 4-grams occur exactly once. A count of one is not a statistic; it records one event. The percentage of unseen test n-grams rises from 7.3% (n=1) to 50.0% (n=2), 81.8% (n=3) and 91.6% (n=4); the number of zero-

probability test sentences out of 925 rises from 645 to 914, 918 and 918. Already at n=2 a typical news sentence is impossible under the model. Memorisation begins between n=2 and n=3: the training table keeps growing by almost one entry per token while coverage of new text collapses.

Cause of the divergence between training and test coverage: the possible n-gram space grows as V^n while data grows only linearly. The training likelihood keeps rising because every training n-gram has non-zero probability (it was counted), while the test likelihood becomes zero as soon as one unseen n-gram appears. Domain shift makes it worse: for fiction the unigram miss rate is 11.0% against 7.3% for news, since vocabulary is domain specific. Templated tickets behave differently: unseen n-grams stay at 0% even at n=4 because the text has only a few hundred distinct 4-grams. Whether a larger n helps therefore depends on how much the language repeats, not only on n. A larger n is not always better: for diverse text it is worse beyond n=2 without smoothing.

2.7 Design decision – 50 MB of tickets, no smoothing

I would choose n = 2 (bigram) for an unsmoothed keyboard. Evidence: on diverse text the trigram already leaves 82% of test trigrams unseen and 918 of 925 sentences impossible, and the 4-gram is worse (92% unseen); the unigram ignores context, so it cannot suggest a next word from the previous one. The bigram is the largest n that keeps coverage near half on 80,000 training tokens, and with 50 MB (roughly ten million words, about 125 times my sample) the bigram table becomes dense for frequent words while the trigram table is still dominated by singletons. For a keyboard an unseen history simply yields no suggestion rather than a crash, so a back-off to the unigram list of frequent words is needed in practice. If the logged tickets are as templated as my synthetic set (0% unseen at n=4), a trigram would be safe, but I should verify this by measuring the unseen rate on a held-out 20% of the real log before accepting n=3. The decision rule is: raise n only while the held-out unseen percentage stays low.

Q3 – Smoothing: Add-One, Backoff and Deleted Interpolation (7 Marks)

3.1 Problem and methods

A speech recogniser proposes candidate transcripts, and the bigram model must give every plausible candidate a non-zero probability, including pairs never seen in training. Let V be the vocabulary size (including <UNK> and </s>), C(h) the history count and N the number of tokens. Four estimators are compared on the same corpus.

- Add-one (Laplace): P(w | h) = (C(h w) + 1) / (C(h) + V). Reconstituted count c* = (C + 1) · C(h) / (C(h) + V). Add-k replaces 1 by a small k tuned on held-out data.
- Backoff with discounting (absolute discount D): for C(h w) > 0, P = (C(h w) − D) / C(h); for unseen w, P = α(h) · P_uni(w) / Σ_{unseen} P_uni(x), where α(h) = D · (number of seen successor types of h) / C(h) is the mass freed by discounting. D = n1 / (n1 + 2 n2) from the training bigram counts (Ney et al., 1994). Katz-style backoff uses Good–Turing discounts; absolute discounting is my stated assumption.
- Deleted interpolation: P(w | h) = λ2 P_ML(w | h) + λ1 P_ML(w) + λ0 / V, with λ0 + λ1 + λ2 = 1 (Jelinek and Mercer, 1980). The λ values are estimated from held-out data by deleted estimation, using EM: the E-step gives each held-out token a posterior responsibility for each component, r_j = λ_j p_j / Σ_i λ_i p_i; the M-step sets λ_j to the normalised sum of responsibilities; 30 iterations. If the history is unseen, the bigram component is dropped and the remaining λ values are renormalised.
- Unigram probabilities are add-0.5 smoothed (so that no vocabulary word has zero probability when the training subset is small). Vocabulary is fixed from the full training set with minimum count 2; rarer words map to <UNK>, so perplexities across training sizes are comparable.

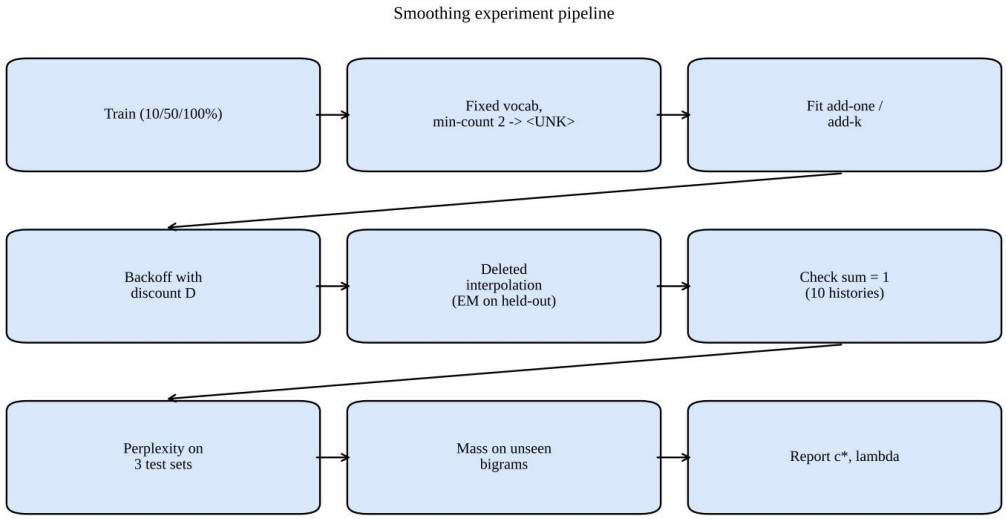


Figure 3.1 – Smoothing experiment pipeline

3.2 Data and predictions

Corpus: Brown news. Sentences are shuffled with seed 42: 80% training (3,698 sentences), 10% held-out for λ and k, 10% test. Training subsets are the first 10%, 50% and 100% of the training sentences. Test conditions: (1) matching test data; (2) other-domain data, 463 Brown fiction sentences; (3) edge case, the matching test with 40% of its words replaced by random strings so that they become <UNK>.

Table 3.1 – Predictions

Prediction (before executing)	Reason
Perplexity order at every size: deleted interpolation ≈ backoff < add-k < add-one	Add-one gives each of V words the same extra count; the two methods that use the unigram redistribute mass in proportion to word frequency
Add-one moves about V / (C(h) + V) of the mass of an average history to unseen bigrams, on the order of 80% with V ≈ 5,400	Most histories have C(h) far smaller than V
All perplexities fall as training grows; the gap between add-one and the others widens	More data gives more seen bigrams; add-one still adds V pseudo-counts per history
λ2 (bigram weight) grows with data, λ0 (uniform) stays near zero	More reliable bigram counts deserve more weight
Domain shift and the unknown-heavy edge set raise the cost of every method; add-one is hurt most	Seen bigrams become rarer

3.3 Verification that probabilities sum to 1

Table 3.2 – Σ over all V words of P(w | h), ten most frequent histories (100% training)

History word	add-one	add-k (k=0.01)	backoff (D=0.75)	deleted interp.
<unk>	1.000000	1.000000	1.000000	1.000000

the	1.000000	1.000000	1.000000	1.000000
,	1.000000	1.000000	1.000000	1.000000
<s>	1.000000	1.000000	1.000000	1.000000
.	1.000000	1.000000	1.000000	1.000000
of	1.000000	1.000000	1.000000	1.000000
and	1.000000	1.000000	1.000000	1.000000
a	1.000000	1.000000	1.000000	1.000000
to	1.000000	1.000000	1.000000	1.000000
in	1.000000	1.000000	1.000000	1.000000

Every method sums to 1.000000 for all ten histories, so each is a proper conditional distribution; the perplexities below are comparable.

3.4 Held-out perplexity

Table 3.3 – matching (news): perplexity and mean probability mass given to unseen bigrams

Method	PP 10%	PP 50%	PP 100%	Unseen mass 10%	50%	100%
add-one	2,048.9	1,073.4	807.2	0.956	0.841	0.761
add-k	886.5	416.8	305.7	0.556	0.394	0.330
backoff	271.9	181.7	155.0	0.633	0.440	0.354
deleted interp.	297.4	197.2	167.6	0.664	0.420	0.319

Table 3.3 – other domain (fiction): perplexity and mean probability mass given to unseen bigrams

Method	PP 10%	PP 50%	PP 100%	Unseen mass 10%	50%	100%
add-one	1,529.6	742.6	545.8	0.950	0.820	0.730
add-k	506.1	281.7	220.8	0.499	0.327	0.266
backoff	164.8	127.3	114.5	0.580	0.399	0.323
deleted interp.	189.1	142.0	126.1	0.637	0.403	0.304

Table 3.3 – edge (40% unknown): perplexity and mean probability mass given to unseen bigrams

Method	PP 10%	PP 50%	PP 100%	Unseen mass 10%	50%	100%
add-one	996.6	447.9	329.5	0.919	0.721	0.598
add-k	322.5	202.1	159.8	0.368	0.245	0.203
backoff	99.2	83.4	78.6	0.523	0.358	0.286
deleted interp.	101.5	82.3	75.9	0.565	0.335	0.242

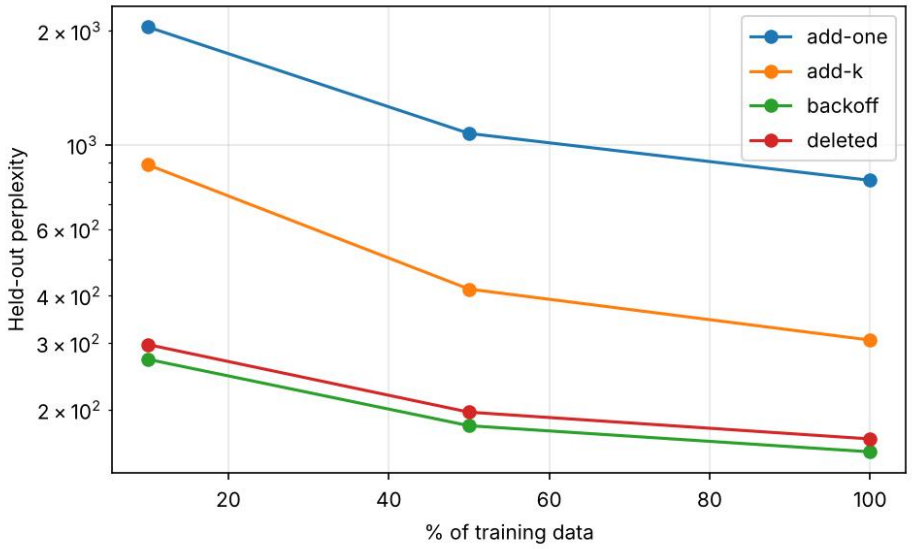


Figure 3.2 – Held-out perplexity versus training size (matching test, log scale)

Table 3.4 – Learned λ values and tuned parameters

Training data	Tokens	λ0 (uniform)	λ1 (unigram)	λ2 (bigram)	k (add-k)	D (backoff)
10%	7,960	0.0054	0.7281	0.2664	0.01	0.882
50%	42,395	0.0370	0.5144	0.4485	0.01	0.798
100%	84,458	0.0163	0.4471	0.5366	0.01	0.754

Table 3.5 – Reconstituted counts c* for seen bigrams (100% training)

Bigram	C	C(h)	Add-one c*	Add-k c*	Backoff c*	Interp. c*
<s> breeding	1	3698	0.809	0.995	0.246	0.595
<unk> homer	2	6246	1.602	1.993	1.246	1.300
) of	3	139	0.100	2.163	2.246	3.241
the wall	5	5116	2.906	4.957	4.246	2.974
, so	10	4170	4.770	9.881	9.246	6.758
, at	20	4170	9.106	19.752	19.246	21.725

3.5 Inquiry

Why does add-one perform poorly? Predicted mass moved to unseen bigrams was 0.81 from the formula V/(C(h)+V) averaged over the test histories, and the measured value was 0.76; roughly three quarters of the probability of every history is spent on words that never followed it. Table 3.5 shows the effect on counts: a bigram seen once (“<s> breeding”) is reduced to c* = 0.81, and a bigram seen 3 times after “)” collapses to c* = 0.10, a cut of about 97%; the discount is not constant but proportional to V / (C(h) + V), which is nearly 1 for rare histories. With V ≈ 5,400 and most histories seen only a few times the model is

dominated by its pseudo-counts. Add-k with a small tuned k (0.01) cuts perplexity from 807 to 306, and backoff / interpolation reduce it to 155 and 168, because they spread the freed mass according to unigram frequency rather than uniformly. Add-one remains acceptable only for tiny vocabularies (classification features, not language models).

How do the λ values change with data? λ2 rises from 0.266 at 10% to 0.449 at 50% and 0.537 at 100%, λ1 falls from 0.728 to 0.447, and λ0 stays below 0.04. As training data grows, bigram counts become more reliable and EM gives the bigram component more weight, exactly as predicted. With 10% of the data the model trusts the unigram more than the bigram (0.73 against 0.27).

What happens when the domain shifts? The absolute perplexity on Brown fiction (add-one 546) is lower than on news, which is a property of the test text (shorter sentences, many function words and fewer rare names), not evidence that the model transfers: its <UNK> rate is higher, and its unseen mass is similar (0.30–0.73). The edge test with 40% unknown words gives the lowest perplexities of all (backoff 79) because <UNK> is a single frequent vocabulary item. This is a warning that perplexity is cheap to improve by hiding words in <UNK>; Q4 develops this point. The ordering of methods is stable across all three conditions, so the choice among smoothing methods is robust to shift, but the absolute quality is not.

Deployment suggestion. For this recogniser re-ranking task I would deploy backoff with discounting ($D \approx 0.75$) when there is at least tens of thousands of tokens of training data: it had the lowest perplexity at all three sizes (272, 182, 155 against 297, 197, 168 for interpolation). My interpolation uses one global λ vector and plain unigram back-off; bucketing λ by history count (Jelinek–Mercer) would likely remove the small gap, which I did not test. My choice would change to interpolation when the model is later extended to a trigram (λ values combine three orders cleanly and can be re-estimated for each domain from a small held-out set), and to add-k with a tuned k only when training data is tiny or the model must be trivial to implement. Add-one should not be deployed for n-gram language models.

Q4 – Entropy and Perplexity: Measuring How Good a Language Model Is (6 Marks)

4.1 Derivations

Entropy. For a discrete distribution p over symbols x, the information content of an outcome is −log2 p(x) bits, and the entropy is its expectation:

$$H(p) = -\Sigma_x\, p(x)\, \log_2 p(x)$$

H is the average number of bits an optimal code needs per symbol. It is 0 for a certain outcome and log2 |X| for the uniform distribution, which is the maximum.

Cross-entropy and per-word cross-entropy. When the true distribution is p but the model is q, the average code length is $H(p, q) = -\Sigma\, p(x) \log_2 q(x) \geq H(p)$, with equality only when q = p (Gibbs inequality). The true p of language is unknown, so for a long test text w1 … wN the Shannon–McMillan–Breiman theorem (stationary, ergodic source) allows estimating it from the text itself:

$$H(W, M) = -\,(1/N)\, \log_2 P_M(w_1 \ldots w_N) = -\,(1/N)\, \Sigma_k \log_2 P_M(w_k \mid w_1 \ldots w_{k-1})$$

Perplexity is defined as PP(W) = P_M(w1 … wN)^(−1/N). Taking log2 gives log2 PP = −(1/N) log2 P_M(w1 … wN) = H(W, M), hence:

$$PP = 2^H$$

Interpretation as a weighted average branching factor: a model that is uniform over b words at every step has P = 1/b each step, so PP = (b^−N)^(−1/N) = b. For a non-uniform model, PP is the geometric mean of the inverse probabilities the model assigned to the words that actually occurred, equal to the size of an equally uncertain uniform choice. A perplexity of 160 means the model is as unsure as if it were choosing among 160 equally likely words at each position.

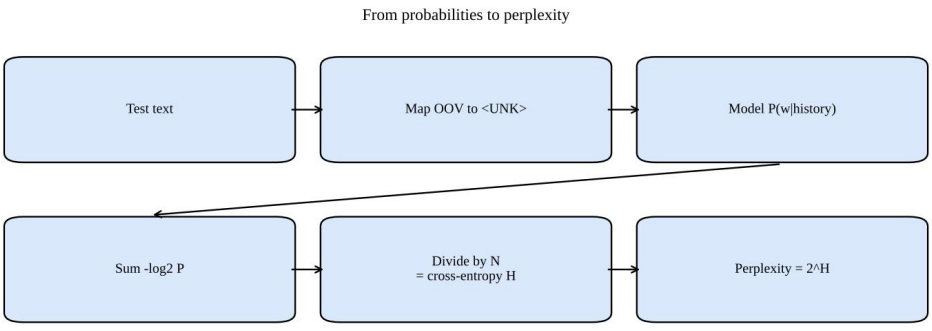


Figure 4.1 – From probabilities to perplexity

4.2 Toy distributions: prediction and check

Predictions made before computing: uniform over 10 symbols has H = log2 10 = 3.32 bits and PP = 10 exactly (the branching factor). A skewed distribution (0.5, 0.2, 0.1, 0.05, 0.05, 0.04, 0.03, 0.01, 0.01, 0.01) has lower entropy; I expected PP between 4 and 6 because the top symbol carries half the mass. Cross-entropy against a mismatched model is never below the entropy of the true source.

Table 4.1 – Toy distributions

Distribution / model	H or H(p,q) (bits)	Perplexity 2^H
uniform-10	3.3219	10.000
skewed-10	2.2656	4.809
dyadic-4	1.7500	3.364
cross: true skewed, model uniform	3.3219	10.000
cross: true uniform, model skewed	4.4922	22.505

Uniform gives exactly 10; the skewed distribution gives 4.81 (inside the predicted band). The dyadic distribution (1/2, 1/4, 1/8, 1/8) has H = 1.75 bits, the same as the average Huffman code length. The two cross-entropy rows show asymmetry: coding a skewed source with a uniform model costs log2 10 = 3.32 bits (perplexity 10, the full vocabulary), whereas coding a uniform source with the skewed model costs 4.49 bits (perplexity 22.5), because the skewed model gives rare symbols probability 0.01.

4.3 Language-model results

Models: interpolated unigram, bigram and trigram (deleted interpolation with EM-estimated λ, the best method class from Q3 for a combined model; I use interpolation here because it generalises directly to trigrams). Training: Brown news 80% (3,698 sentences); λ fitted on the held-out 10%. Three test texts of equal sentence count: in-domain news (463 sentences), out-of-domain Brown fiction, and an edge case, the in-domain test with the words of each sentence randomly shuffled (seed 42). Vocabulary: words with count ≥ 2 (5,447 types including <UNK> and </s>).

Table 4.2 – Entropy and perplexity of three models on three texts

Model	Test text	H (bits/word)	Perplexity	Tokens	<UNK> tokens	Learned λ
unigram	in-domain	8.40	336.7	10,455	1,205 (11.5%)	0.00, 1.00
unigram	out-of-domain	7.80	223.2	7,774	1,174 (15.1%)	0.00, 1.00
unigram	shuffled	8.40	336.7	10,455	1,205 (11.5%)	0.00, 1.00
bigram	in-domain	7.39	167.6	10,455	1,205 (11.5%)	0.02, 0.44, 0.54
bigram	out-of-domain	6.98	126.2	7,774	1,174 (15.1%)	0.02, 0.44, 0.54
bigram	shuffled	8.92	484.4	10,455	1,205 (11.5%)	0.02, 0.44, 0.54

trigram	in-domain	7.34	161.7	10,455	1,205 (11.5%)	0.02, 0.44, 0.48, 0.06
trigram	out-of-domain	6.96	124.6	7,774	1,174 (15.1%)	0.02, 0.44, 0.48, 0.06
trigram	shuffled	8.89	473.6	10,455	1,205 (11.5%)	0.02, 0.44, 0.48, 0.06

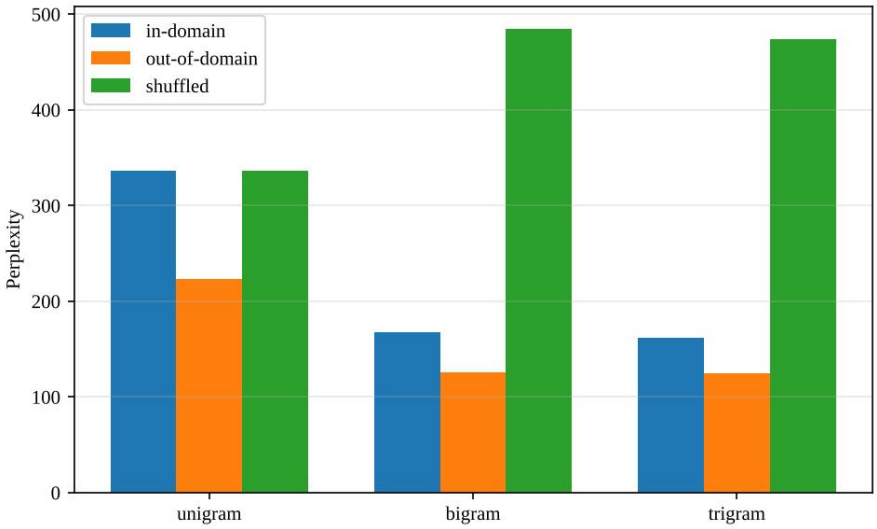


Figure 4.2 – Perplexity by model and test text (min-count 2 vocabulary)

4.4 Effect of vocabulary restriction and <UNK> handling (bigram)

Table 4.3 – Bigram, in-domain test: vocabulary size and <UNK> treatment

Min count	Vocabulary V	Unknown tokens	# unknown	% unknown	H	Perplexity
2	5,447	scored as <UNK>	1,205	11.5%	7.39	167.6
1	11,693	scored as <UNK>	750	7.2%	8.93	489.1
2	5,447	skipped (not scored)	1,205	11.5%	7.88	236.3
5	2,094	scored as <UNK>	2,054	19.6%	6.26	76.7
5	2,094	skipped (not scored)	2,054	19.6%	7.22	149.2
10	985	scored as <UNK>	2,822	27.0%	5.34	40.4
10	985	skipped (not scored)	2,822	27.0%	6.64	99.7

The same model family on the same test text yields perplexities from 40 to 489 depending only on the vocabulary rule. Restricting the vocabulary to words with count ≥ 10 (V = 985) makes perplexity 40 although the model is no better: 27% of the test tokens are now <UNK>, which is one cheap, frequent “word”. Treating <UNK> differently (skipping those tokens) changes the number from 168 to 236 for min-count 2, because the model is scored on a harder subset.

4.5 Inquiry

Why can two perplexities not be compared when vocabularies differ? Perplexity is an average branching factor over a particular set of events. A model over V1 = 985 words and a

model over V2 = 11,693 words are predicting different tasks: the first collapses all rare words into <UNK> and is only asked to predict that symbol, the second must also name the rare word. Table 4.3 shows 40 versus 489 for the same text and nearly the same model. A claim by two teams is comparable only if they use the same test text, the same tokenisation, the same vocabulary and the same <UNK> rule, or if both convert to a closed-vocabulary measure such as bits per character, which has a fixed alphabet. This is why I fixed the vocabulary across training sizes in Q3.

Why does lower perplexity not always give better downstream performance? Perplexity measures the average log-probability of the word that occurred, not the ranking errors a recogniser or translator makes, and it is dominated by frequent words. A model can lower perplexity by improving the prediction of “the” and “of” while it does not help distinguish “recognise speech” from “wreck a nice beach”. It can also lower perplexity by hiding rare words in <UNK>, as shown above, which would hurt a task that needs to output those rare words (names in speech recognition). Downstream metrics (word error rate, BLEU, tagging accuracy) must be measured directly; perplexity is a cheap proxy that is useful for comparing models under the same conditions.

What does the shuffled-text result reveal? The unigram perplexity is identical for the original and shuffled text (336.7 and 336.7), as it must be, since a unigram model sees only the multiset of words. The bigram perplexity rises from 168 to 484 (×2.9) and the trigram from 162 to 474, so the models capture word order. The bigram model after shuffling (484) is worse than the unigram model (337) because its λ values were fitted for ordered text and trust the bigram component with weight 0.54; on disordered text those bigram predictions are wrong. The in-domain gain from the bigram is 50% and the trigram adds only a further 4%, which is the sparsity of Q2 appearing again: with 80,000 tokens, most of the context information is already in the previous word. The models capture short-range local statistics, not meaning: the shuffled text has the same words and the same unigram probability, and only local order separates it.

Q5 – Word Classes and Tagset Design

5.1 Open and closed classes with distributional evidence

Open classes accept new members readily (new products, verbs and brand names appear every year); closed classes have a small, stable membership that rarely changes and consist mainly of function words.

Table 5.1 – Classes justified by distribution

Class	Open / closed	Distributional evidence (what it follows and precedes)	Inflection
Noun	Open	Follows determiners and adjectives (the network, a slow connection); is the head of a subject, "___ is down"	Plural -s; possessive 's
Verb	Open	Follows subjects, modals and "to" (it can drop, to cancel); takes objects: "___ my plan"	-s, -ed, -ing, irregular (broke, broken)
Adjective	Open	Between determiner and noun (a stable signal); after "very" or "is" (very slow)	-er, -est (faster, fastest)
Adverb	Open	Modifies verbs, adjectives or sentences, often movable (quickly, today); follows "very" (very badly)	-ly; -er/-est for some
Determiner	Closed	Precedes nouns and blocks another determiner (the, a, this, my)	None
Preposition	Closed	Precedes a noun phrase (in, on, via, for); never takes -ed or -s	None
Pronoun	Closed	Replaces a noun phrase; has case forms (I, me, my)	Case, no -s plural
Conjunction	Closed	Links words or clauses (and, but, because)	None
Modal / auxiliary	Closed	Precedes a base verb and inverts with the subject in questions (can it work?)	None (no -s, no -ing)
Particle	Closed	Follows a verb and can follow its object: "turn the router off" (RP) versus a preposition, which cannot move	None

5.2 Three tagsets of different granularity

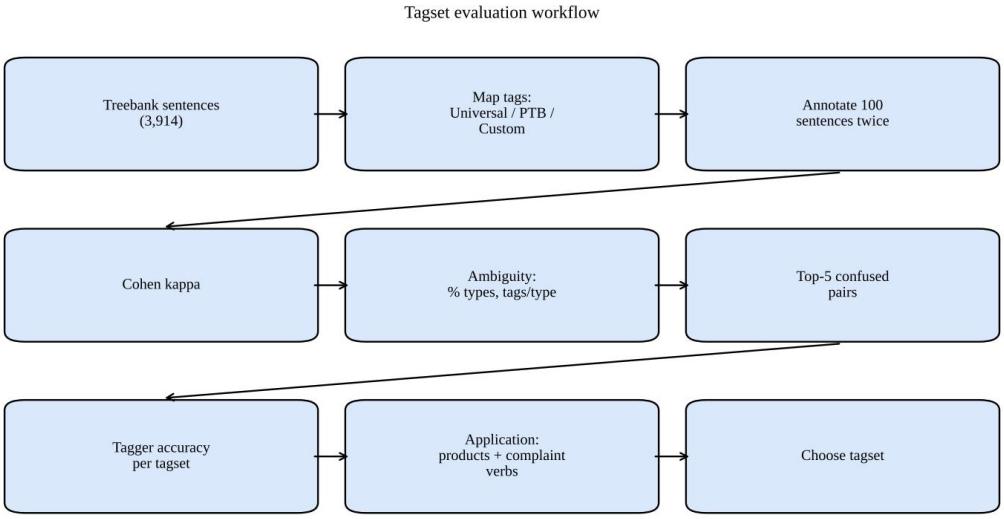


Figure 5.1 – Tagset evaluation workflow

Coarse: Universal POS, 12 tags (NOUN, VERB, ADJ, ADV, PRON, DET, ADP, NUM, CONJ, PRT, ., X) from Petrov, Das and McDonald (2012). Medium: the Penn Treebank tagset, 45 tags observed in the sample (Marcus et al., 1993), which separates NN/NNS/NNP/NNPS, six verb forms and so on. Fine custom tagset for customer-support chat. The 20 custom tags that I evaluated are derived by mapping from Treebank tags; the additional chat-specific tags (EMO, URL/CODE, ORDER_ID and TAMIL words) are justified in the second table and are used in the hand-annotated examples.

Table 5.2 – Custom tags and why each was added

Custom tag	Replaces / splits	Justification for this domain
PROD	NNP, NNPS	Proper nouns in support chats are product and plan names (JioFiber, Rs.499 plan); the extraction task needs them as a class of their own
ABBR	Upper-case NNP ≤ 4 letters	Network and device abbreviations (SIM, APN, OTP, LTE) behave differently from product names and have no suffix cues
V_CMPL	VB* when the stem is in a complaint-verb lexicon (fail, crash, drop, lag, freeze, disconnect, cancel, block, refuse, delay, cut, lose, stop)	Required by the application: finding complaint verbs
V_GEN	All other VB*	Remaining verbs; tense kept out because the extraction task does not need it, which reduces confusion between VB/VBP and VBD/VBN
MODAL, TO	MD, TO	Closed classes whose distribution is distinct and which anchor verb-tag decisions
NOUN	NN, NNS	Singular and plural merged: number is rarely needed for extraction and NN/NNS adds

		ambiguity
ADJ, ADV, NUM, PRON, DET, PREP, CONJ, WH, PART, INTJ, SYM, PUNCT, TAMIL_MIX, OTHER	JJ*, RB*, CD, PRP*, DT/EX/PDT, IN, CC, W*, RP/POS, UH, SYM, punctuation, FW	Standard classes at moderate granularity; FW is repurposed as TAMIL_MIX for code-mixed words
Tags added for chat, not in the Treebank	EMO, CODE (4G, OTP-1234), URL, HASH/MENTION	Emoticons carry sentiment, codes are neither words nor numbers, URLs must be skipped by taggers; justified by the mix described in the problem and evaluated on the chat sample below

Table 5.3 – Hand-annotated domain examples (illustrating the added tags)

Chat sentence	Hand annotation with the custom tagset
Net speed 4G isn't working :(	Net/NOUN speed/NOUN 4G/CODE is/V_GEN n't/PART working/V_CMPL :(/EMO
Jio SIM keeps dropping the call da	Jio/PROD SIM/ABBR keeps/V_GEN dropping/V_CMPL the/DET call/NOUN da/TAMIL_MIX
Please cancel my Rs.499 plan	Please/INTJ cancel/V_CMPL my/PRON Rs/ABBR 499/NUM plan/NOUN
OTP not received semma late	OTP/ABBR not/PART received/V_CMPL semma/TAMIL_MIX late/ADV
App crashes whenever I recharge via e-wallet	App/NOUN crashes/V_CMPL whenever/CONJ I/PRON recharge/V_GEN via/PREP e-wallet/NOUN
Visit https://example.in/help for refund	Visit/V_GEN https://example.in/help/URL for/PREP refund/NOUN

5.3 Predictions

Agreement (kappa) should fall as granularity increases, in the order coarse > custom > Penn, because each added distinction (NN versus NNP, VB versus VBP, JJ versus NN) is a place where two annotators can disagree. Ambiguity (percentage of word types with more than one tag, and average tags per type) should rise in the same order, since more tags give a word more ways to be labelled. Tagger accuracy should fall as the tagset grows because the per-tag evidence thins. The custom set sits between the others since it merges the six Penn verb tags into two but splits proper nouns.

5.4 Results (Penn Treebank sample, 3,914 sentences, 100,676 tokens; 100 sentences annotated)

Table 5.4 – Agreement, ambiguity and tagger accuracy by tagset

Tagset	# tags	Cohen's κ	% agreement	% ambiguous types	Avg tags / type	HMM accuracy (held-out)
Coarse (Universal, 12)	12	0.9423	95.19	6.88	1.076	94.87%
Penn Treebank (46)	45	0.9347	93.96	13.41	1.161	94.03%

Custom domain tagset	20	0.9373	94.45	9.99	1.115	94.41%

Table 5.5 – Five most-confused tag pairs (gold → other annotation, count)

Tagset	Confused pair 1	Pair 2	Pair 3	Pair 4	Pair 5
Coarse (Universal, 12)	NOUN → ADJ (19)	VERB → NOUN (14)	ADJ → NOUN (13)	NOUN → VERB (11)	ADJ → VERB (6)
Penn Treebank (46)	NNP → NN (15)	NN → NNP (10)	JJ → NN (9)	NN → JJ (8)	VBP → VB (8)
Custom domain tagset	PROD → NOUN (18)	NOUN → PROD (12)	NOUN → V_GEN (11)	V_GEN → NOUN (11)	NOUN → ADJ (9)

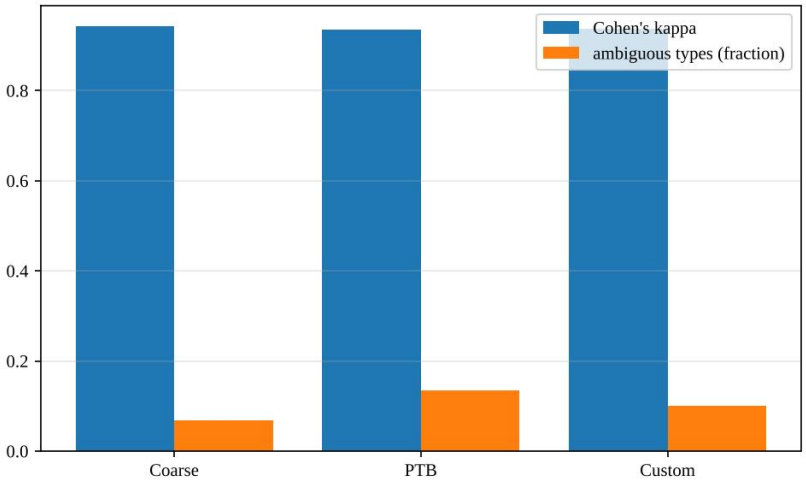


Figure 5.2 – Agreement and ambiguity per tagset

5.5 Inquiry

The prediction on ordering is supported for ambiguity, and the ordering of kappa matches (coarse 0.942 > custom 0.937 > Penn 0.935), but the kappa differences are small (under one point). Ambiguity rises clearly from 6.9% of types (coarse) to 10.0% (custom) and 13.4% (Penn), and average tags per type from 1.076 to 1.161. Tagger accuracy follows: 94.9%, 94.4%, 94.0%. The small kappa gap is because my second annotator is a statistical tagger trained on 3,000 sentences of the same genre, which errs on the same hard cases whatever the labels; a human annotator on noisy chat text is likely to show a larger granularity effect, which my own re-annotation would test.

Hardest classes: the confusion pairs are consistent across tagsets and are all boundary cases between open classes: noun versus adjective (JJ/NN, “stock market”, “bank loan”, where a noun modifies a noun), noun versus verb (“the plans”, “to plan”; a word with both uses), and proper versus common noun (NNP/NN) for capitalised words at sentence start and product-

like names. In the custom set PROD ↔ NOUN is the top confusion, which is dangerous because PROD is the class the application needs. The reason is that the distinguishing evidence is not local syntax but a lexical decision ("is this a product name?"), which depends on the world. Particle versus preposition (RP/IN) and the VBG/NN gerund also appear in the literature but are infrequent in this sample.

What is lost by a coarse tagset, and what does a fine one cost? The coarse set merges NNP with NN, so "find the product name" cannot be answered from tags; it also hides tense and number, which a parser or a rule such as "complaint verb in past tense" needs. The fine set separates these but yields more ambiguous types, more confusion pairs, smaller counts per tag (more sparsity in the HMM) and a heavier annotation guideline. Annotation cost grows with the number of distinctions that must be remembered.

5.6 Protocol for my own re-annotation, and the decision

Protocol for the human annotation required by the assignment: annotate 100 chat-like sentences with each tagset, wait at least 48 hours, re-annotate blind in a fresh file, compute Cohen's κ with the same script (q5.py, function kappa) and list disagreements. The proxy annotator in this report should be replaced by those labels; the scripts take two lists of tags.

Design decision: for an extraction application that must find product names and complaint verbs I select the custom tagset, with PROD, ABBR and V_CMPL kept and tense and number merged. Evidence: its agreement (94.5%) is slightly higher than Penn Treebank's (94.0%) and its ambiguity is lower (10.0% against 13.4% of types) while it keeps the two distinctions the Universal tagset cannot make: a product-name tag and a complaint-verb tag. Its tagger accuracy (94.4%) is higher than Penn's. The coarse set has the best agreement but cannot answer the application's question. The risk is the PROD/NOUN confusion, to be handled by a product-name gazetteer.

Q6 – Rule-Based, Stochastic and Transformation-Based Tagging

6.1 Setup

Headlines such as “Time flies”, “Saw the saw” and “They can fish” are ambiguous: time can be NOUN or VERB, flies NOUN or VERB, saw NOUN or VERB, can MODAL, NOUN or VERB. Corpus: Brown Corpus “news” category (Francis and Kučera, 1979) with the universal tagset (12 tags), 4,623 sentences, shuffled with seed 42 and split 80/20 (3,698 train, 925 test, 19,794 test tokens). Out-of-domain: the first 1,500 Brown “fiction” sentences. Training sizes: 10%, 50% and 100% of the training sentences. Known / unknown: a test word is known if it occurs in the training subset used by that model.

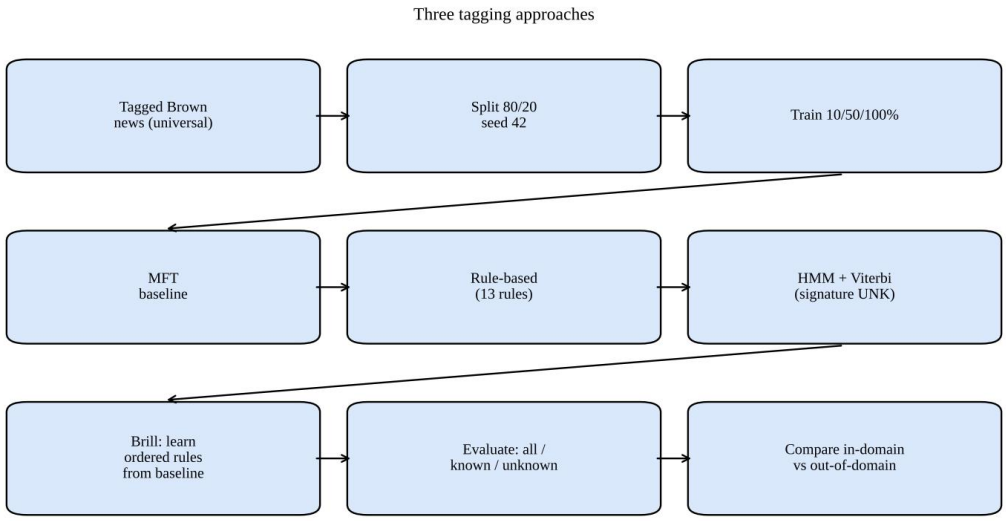


Figure 6.1 – Three tagging approaches and their evaluation

Table 6.1 – The four taggers

Tagger	Description
MFT baseline	Each known word gets its most frequent training tag; unknown words get NOUN
Rule-based	Lexicon of most frequent tags plus 13 ordered hand-written constraint rules (below); R2–R8 handle unknown words using suffix and capitalisation, R9–R13 resolve known ambiguity from the previous tag or word
HMM + Viterbi	Bigram HMM: P(tags) = Π P(ti | ti−1), emission P(wi | ti). Transitions add-0.01 smoothed. Known words: relative frequency of the word given the tag, with floor 1e-8 for unseen pairs. Unknown words: signature model P(signature | tag), estimated from the training words that occur once (hapax), with 14 signature classes (digit, capitalised, hyphen, suffixes -ing -ed -ly -ion -ness -ous -al -ble -ic -ive -er -est -s, other)
Brill (TBL)	Initial tagger is a lexicon baseline trained on the first 60% of the training subset; ordered transformation rules are learned on the remaining 40%, which contains unknown words, from 11 templates (previous / next / two previous tags, previous / next word, word + previous tag, 3-letter suffix, capitalisation, and combinations); minimum score 3, at most 40 rules

Table 6.2 – Hand-written rules of the rule-based tagger

Rule	Condition (applied in order)	Action
R1	Token is punctuation	Tag "."
R2	Unknown word containing a digit	NUM
R3	Unknown, capitalised, not first word	NOUN (proper noun)
R4	Unknown, ends in -ing	VERB
R5	Unknown, ends in -ed	VERB
R6	Unknown, ends in -ly	ADV
R7	Unknown, ends in -ous/-ful/-ive/-al/-able/-ible/-ic/-less	ADJ
R8	Unknown, ends in -s, previous tag DET or ADJ	NOUN
R9	Previous word is "to" and VERB is a seen tag of the word	VERB
R10	Previous tag DET, current tag VERB, NOUN is a seen tag	NOUN
R11	Previous tag ADJ, current VERB, NOUN seen	NOUN
R12	Previous tag PRON, current NOUN, VERB seen	VERB
R13	Previous tag ADP, current VERB, NOUN seen, word not ending in -ing	NOUN

6.2 Predictions

Table 6.3 – Predictions made before the discussion of results

Condition	Predicted accuracy ranking (best → worst)	Reason
Known words	HMM > rule-based ≈ Brill > MFT	HMM uses tag context to choose among tags of an ambiguous word; MFT has none
Unknown words	Rule-based ≈ Brill > HMM > MFT	Suffix and capitalisation rules encode strong morphological cues; the HMM signature model is coarse
Out-of-domain	Same order, all lower; unknown words have the largest drop	New domain means more unknown words and shifted tag statistics
Most confused pairs	VERB↔NOUN, ADJ↔NOUN, ADP↔PRT	Same word forms serve in both classes (plan, cut, back, up)
Small data (10%)	Rule-based and Brill degrade least; HMM transition estimates suffer	Rules need little data; HMM needs counts

6.3 Accuracy results

Table 6.4 – Accuracy, test = news

Train data	Tagger	Overall %	Known %	Unknown %	Unknown / total tokens
10%	MFT baseline	86.24	95.34	59.41	5,016 / 19,794
10%	Rule-based	91.23	95.49	78.69	5,016 / 19,794
10%	HMM+Viterbi	90.67	96.52	73.44	5,016 / 19,794
10%	Brill (TBL)	88.40	93.94	72.05	5,016 / 19,794
50%	MFT baseline	91.23	95.52	58.10	2,265 / 19,794

50%	Rule-based	93.89	95.85	78.72	2,265 / 19,794
50%	HMM+Viterbi	94.29	96.94	73.77	2,265 / 19,794
50%	Brill (TBL)	92.68	95.17	73.42	2,265 / 19,794
100%	MFT baseline	92.80	95.69	57.89	1,515 / 19,794
100%	Rule-based	94.71	96.05	78.48	1,515 / 19,794
100%	HMM+Viterbi	95.09	97.14	70.36	1,515 / 19,794
100%	Brill (TBL)	93.75	95.49	72.81	1,515 / 19,794

Table 6.4 – Accuracy, test = out-of-domain

Train data	Tagger	Overall %	Known %	Unknown %	Unknown / total tokens
10%	MFT baseline	81.64	93.77	47.91	5,940 / 22,465
10%	Rule-based	87.74	93.88	70.66	5,940 / 22,465
10%	HMM+Viterbi	88.32	95.60	68.08	5,940 / 22,465
10%	Brill (TBL)	85.00	92.09	65.27	5,940 / 22,465
50%	MFT baseline	87.62	94.15	51.09	3,408 / 22,465
50%	Rule-based	91.50	94.46	74.91	3,408 / 22,465
50%	HMM+Viterbi	92.52	95.90	73.65	3,408 / 22,465
50%	Brill (TBL)	90.61	93.80	72.74	3,408 / 22,465
100%	MFT baseline	89.45	94.32	52.50	2,617 / 22,465
100%	Rule-based	92.58	94.58	77.34	2,617 / 22,465
100%	HMM+Viterbi	93.23	95.90	72.98	2,617 / 22,465
100%	Brill (TBL)	91.78	94.20	73.48	2,617 / 22,465

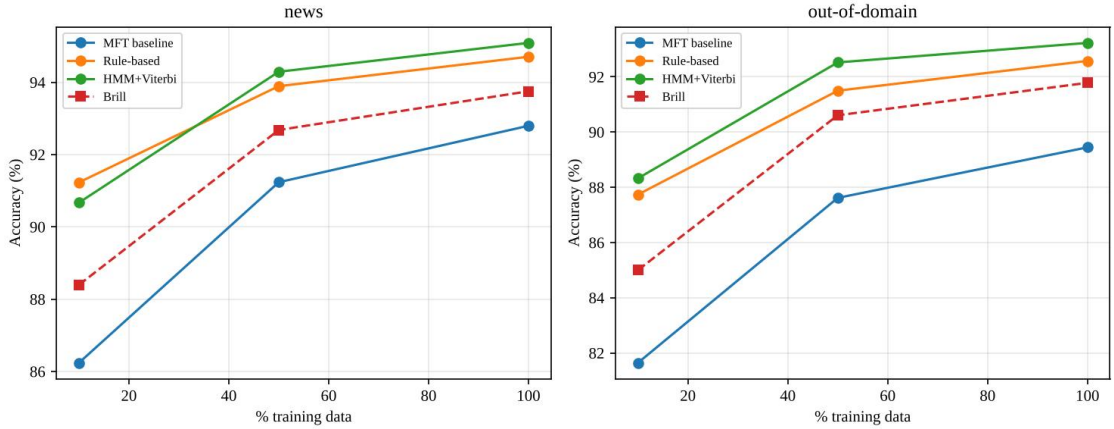


Figure 6.2 – Accuracy versus training size, in-domain (left) and out-of-domain (right)

Note on Brill known/unknown counts: its "known" label uses the vocabulary of the whole training subset, not just the 60% used for its lexicon, so its unknown-word column is computed on the same test tokens as the other taggers.

6.4 Confusion, Viterbi counts and Brill transformations

Table 6.5 – Most frequent errors (gold → predicted : count), 100% training

Tagger / test	Top 1	Top 2	Top 3	Top 4	Top 5
MFT (news)	VERB → NOUN:	ADJ → NOUN:	ADP → PRT: 176	NOUN → VERB:	NUM → NOUN:

	432	235		108	81
MFT (out-of-domain)	VERB → NOUN: 851	ADJ → NOUN: 384	ADP → PRT: 228	ADV → ADJ: 118	ADV → NOUN: 109
Rule (news)	VERB → NOUN: 217	ADP → PRT: 176	ADJ → NOUN: 129	NOUN → ADJ: 70	NOUN → VERB: 66
Rule (out-of-domain)	VERB → NOUN: 294	ADJ → NOUN: 270	ADP → PRT: 228	ADV → ADJ: 118	NOUN → VERB: 85
HMM (news)	VERB → NOUN: 129	NOUN → VERB: 83	NOUN → ADJ: 77	ADJ → NOUN: 73	NOUN → DET: 66
HMM (out-of-domain)	VERB → NOUN: 168	ADJ → NOUN: 151	NOUN → VERB: 116	NOUN → ADJ: 82	ADV → ADJ: 72

Viterbi transitions evaluated: for a sentence of n words and T tags, Viterbi evaluates T for the first word and T^2 for each later word, $(n-1)T^2 + T$ in total. With T = 12 the 19,794-token news test set (925 sentences) needed 2,728,236 transition evaluations, the same at every training size because the tagset and test text are fixed (138 per token, about $T^2 = 144$). The fiction test set needed 3,036,960.

Table 6.6 – Rule activity (100% lexicon)

Rule	Times fired (news test)
R1 punct->.	2,315
R2 digit->NUM	130
R3 capitalised->NOUN	388
R4 -ing->VERB	103
R5 -ed->VERB	124
R6 -ly->ADV	44
R7 adj suffix->ADJ	60
R8 DET+..s->NOUN	87
R9 to+X->VERB	41
R10 DET+VERB?->NOUN	42
R11 ADJ+VERB?->NOUN	15
R12 PRON+NOUN?->VERB	7
R13 ADP+VERB?->NOUN	18

Table 6.7 – Brill rule learning

Training data	Transformations learned	Training errors before	after	Initial lexicon tokens	Rule-learning tokens
10%	18	474	330	4,545	3,046
50%	40	1566	1069	24,147	16,399
100%	40	2575	1791	48,525	32,235

Table 6.8 – Accuracy gain after each prefix of the learned rules (100% training)

# rules applied	News accuracy %	News unknown %	Fiction accuracy %
0	91.62	57.89	88.05
1	91.86	57.89	88.34
2	92.08	61.12	88.77

3	92.23	62.51	88.98
5	92.45	64.29	89.67
8	92.71	65.02	89.85
12	92.93	67.00	90.17
20	93.36	69.70	90.98
30	93.58	70.50	91.27
40	93.75	72.81	91.78

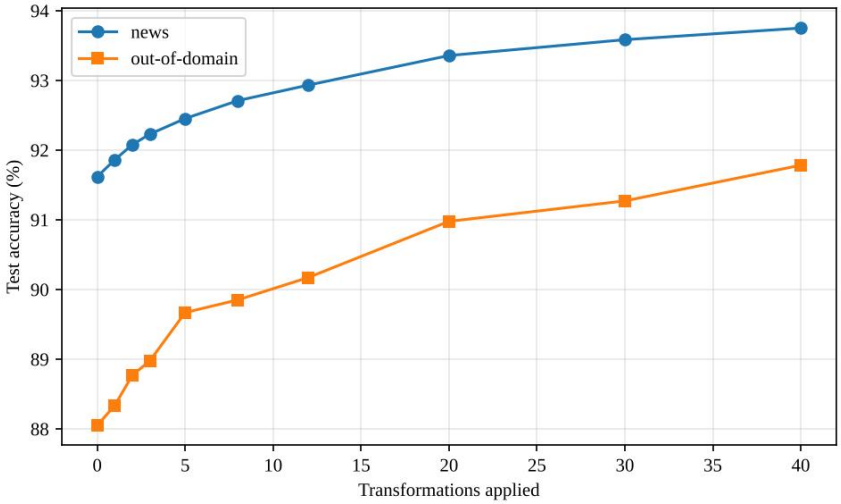


Figure 6.3 – Brill learning curve on test data (100% training; 40 rules is the limit I set)

Table 6.9 – Learned rules in order

#	First learned transformations (100% training)
1	PRT->ADP if Pos:DET@[1]
2	NOUN->VERB if Pos:DET@[1]
3	NOUN->VERB if Suf:ted@[0]
4	NOUN->VERB if Cap:False@[0] & Pos:PRON@[-1]
5	NOUN->VERB if Cap:False@[0] & Pos:PRT@[-1]
6	PRT->ADP if Pos:NOUN@[1]
7	VERB->NOUN if Word:which@[1]
8	NOUN->VERB if Suf:ing@[0] & Pos:ADP@[-1]
9	NOUN->VERB if Suf:red@[0]
10	NOUN->ADJ if Suf:ous@[0]

6.5 Inquiry

Accuracy ranking (100% data, news): HMM 95.09%, rule-based 94.71%, Brill 93.75%, MFT 92.80%. On known words the order was as predicted (HMM 97.14% best, then rule-based 96.05%, MFT 95.69%). On unknown words the prediction held partly: rule-based was best (78.5%), but Brill (72.8%) was above the HMM (70.4%), and my prediction that Brill would roughly tie the rule-based tagger on known words was too optimistic: Brill stopped at my 40-

rule cap while still improving (Figure 6.3), and its initial lexicon used only 60% of the training data. Out-of-domain accuracy was lower for every tagger, and the largest drop was on unknown words, as predicted.

Errors shared by all three approaches: VERB ↔ NOUN, ADJ ↔ NOUN and ADP → PRT top every confusion list. They are shared because the information that decides them is not available from local tag context or suffixes: whether “plan” is a noun or a verb in a headline depends on the sentence’s structure (“Bank plans merger”), and ADP versus PRT depends on whether the word attaches to a verb or a noun phrase (“about”, “up”). Both are attachment decisions that a bigram tag context and word-level rules cannot see. The rule-based tagger leaves ADP → PRT at 176 errors on news exactly as MFT does, because none of my rules addresses it, whereas the HMM has fewer than the baseline.

Behaviour with little data (10% of the training sentences, about 8,000 tokens; Table 6.4): the rule-based tagger lost the least (91.2% at 10% against 94.7% at 100%, a 3.5-point drop) because its rules are written by hand, not learned; the HMM lost 4.4 points; MFT lost 6.6 points; Brill gained from each rule but starts from a weak lexicon. Transparency: the rule-based tagger is the most transparent (13 readable rules and counts of firings), Brill is next (an ordered list of human-readable transformations, Table 6.9), and the HMM is the least (probability tables) although it is the easiest to analyse mathematically. Most accurate overall: the HMM with Viterbi in-domain; the rule-based tagger on unknown words.

Recommendation for a low-resource language with 2,000 tagged sentences (roughly 40,000 tokens, comparable to my 50% subset): I recommend a hybrid, an HMM tagger with a suffix-signature unknown-word model, with a short list of hand-written rules for the commonest confusions, and Brill-style rule learning as an error-correction layer. Evidence from my own runs: at 50% (about 1,850 sentences) the HMM reached 94.3% in-domain and 92.5% out-of-domain, the best overall, and the rule-based tagger reached 93.9% and 91.5%. The rule-based tagger is the safest when the corpus is even smaller or morphological resources exist, but writing rules for a new language needs a linguist, so the HMM + suffix model is a better default, and Brill rules on the HMM output can add transparency. A caveat: English suffix cues differ from those of agglutinative languages; signature classes would have to be redesigned.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Selected solution: the stochastic HMM tagger of Q6 with Viterbi decoding. Limitation chosen from the list: Viterbi evaluates a number of transitions that grows with sentence length and with the square of the tagset size. The redesign is not a neural network; it uses properties of the data the original ignores.

A. Diagnose the bottleneck (2 marks)

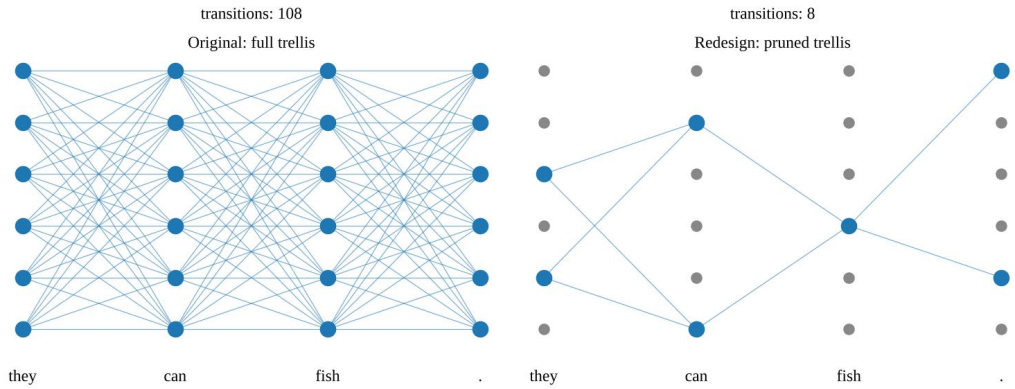


Figure 7.1 – Full trellis (left, T states at every position) versus pruned trellis (right) for "they can fish ."

Where the effort goes. Viterbi fills a trellis of n positions by T states. For each state at position i it takes the maximum over all T previous states, so each step costs T^2 transition evaluations and a sentence costs $(n-1)T^2 + T$. In Q6 the universal tagset (T = 12) needed 2,728,236 transition evaluations for the 925 news test sentences, 138 per token. Using the Brown tagset with its suffix variants removed, which I use for this question because a real IE application would want a finer tagset (Q5), T grows to 98 and the same sentences need 181,308,526 evaluations, 9,160 per token, 66 times more per token. These counts follow T^2 exactly ($98^2 = 9{,}604$ per word), and do not depend on the words.

Which counted quantity grows: transitions per word (T^2) and total transitions ($n\,T^2$); with sentence length n the cost is linear in n per sentence, and the figure below shows the original rising from about 42,000 per sentence for sentences under 10 words to over 575,000 for 60-word sentences. Wasted information: a known word such as "the" has been seen with only one or two tags in 100,000 tokens, yet the original evaluates all T = 98 states for it, with an emission probability fixed at the floor 1e-8 for all but the one or two tags seen with it; those paths can never win. The computation is repeated and unnecessary, because the lexicon already says which tags are possible. Which design characteristic causes it: the decoder treats

the tag set as the state set at every position, ignoring the sparsity of the word–tag relation (most words have one tag; the average number of tags seen per known test token is about 1.4).

B. Design objective (2 marks)

Before proposing the solution: reduce the number of transitions evaluated by at least 95% on the held-out news test (target: below 9 million from 181 million) and by at least 90% on out-of-domain text, while keeping overall tagging accuracy within 0.3 percentage points of the original and unknown-word accuracy within 1 point. Cost is measured by counted transitions, not run-time.

C. Revised strategy (4 marks)

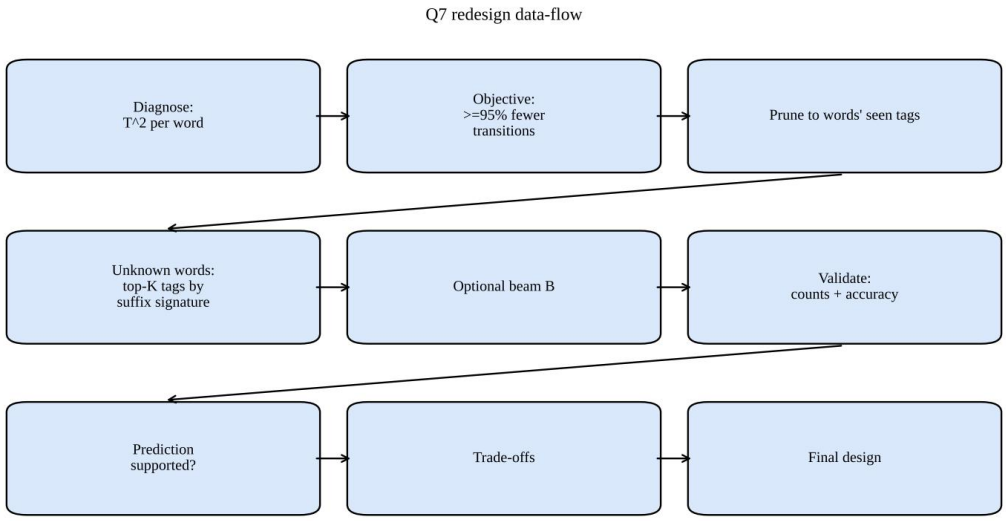


Figure 7.2 – Data-flow of the redesign

What changes, and what it exploits: replace the full state set at each position by a candidate set built from information already in the model. (1) For a known word, the candidates are only the tags it was seen with in training (the tag dictionary). This exploits the sparsity of the word–tag relation, since words are mostly unambiguous. (2) For an unknown word, the candidates are the K most probable tags for its signature class (suffix, capitalisation, digits), taken from the hapax-word statistics that the model already holds; K = 10 is the default. This exploits the fact that suffix and shape restrict a word to a few open classes. (3) An optional beam keeps only the best B states of the previous position. Transition cost per word becomes $|C(i-1)| \times |C(i)|$ instead of T^2.

```
function VITERBI_PRUNED(words, K, B):
  for i in 0..n-1:
    if word_i in lexicon: C[i] = tags_seen_with(word_i)
```

```
    else:              C[i] = top_K_tags(signature(word_i))
  delta = logStart[C[0]] + logEmit(word_0)[C[0]];  count += |C[0]|
  for i in 1..n-1:
    if B and |C[i-1]| > B: keep the B best states of delta
    M = delta[:, None] + logTrans[C[i-1], C[i]]        # |C(i-1)| x |C(i)| only
    count += |C[i-1]| * |C[i]|
    back[i] = argmax_row(M);  delta = max_row(M) + logEmit(word_i)[C[i]]
  return backtrace(delta, back)
```

Avoided storage and operations: the T × T matrix scan per word (reduced to a small sub-matrix), and emission lookups for impossible tags. New state and complexity: a tag dictionary (word → seen tags; already implied by the emission table) and a signature → top-K tag table; two parameters K and B. New trade-offs: if the correct tag for a known word was never seen with it in training, the pruned decoder cannot output it (the original could, through the 1e-8 floor), and a too small K hurts unknown words; the beam B can prune the best path early.

D. Prediction before implementing (1 mark)

Prediction: tag-dictionary pruning with K = 10 will cut transitions by more than 99% (the average candidate set is about 1.4 for known words, so a known–known step costs about 2 evaluations, while the 7.7% of tokens that are unknown add up to 10 × 10 = 100 for each pair that involves them), giving about 100,000 or fewer in total on the news test instead of 181 million, with overall accuracy unchanged to within ±0.2 points. Known-word accuracy may fall slightly (correct tags never seen with the word are lost) and unknown-word accuracy should be unchanged or better (a restricted candidate set removes implausible tags). Smaller K (5) should lose unknown-word accuracy; a beam of 3–5 should save a further 20% of transitions at a small accuracy cost.

E. Implementation and validation

Prototype: the HMM class with a prune flag, K and B parameters (taggers.py). Same trained model and test sets for all rows; original versus redesign. Tags: 98; training sentences 3,698; test sentences 925 (news), 1,500 (fiction).

Table 7.1 – Original versus redesign, test = news

Configuration	Transitions	Per word	Reduction	Accuracy %	Known %	Unknown %
Original (full Viterbi)	181,308,526	9,159.8	–	94.31	96.64	66.20
Tag-dictionary pruning	86,195	4.4	99.952%	94.31	96.53	67.52
Pruning + beam 5	70,155	3.5	99.961%	94.13	96.36	67.33
Pruning + beam 3	62,833	3.2	99.965%	94.11	96.33	67.26
Pruning, K=5 unk tags	56,930	2.9	99.969%	93.67	96.48	59.80

Pruning, K=20 unk tags	164,225	8.3	99.909%	94.22	96.58	65.74

Table 7.1 – Original versus redesign, test = out-of-domain

Configuration	Transitions	Per word	Reduction	Accuracy %	Known %	Unknown %
Original (full Viterbi)	201,494,860	8,969.3	–	91.51	95.41	61.90
Tag-dictionary pruning	132,250	5.9	99.934%	91.45	95.28	62.40
Pruning + beam 5	101,910	4.5	99.949%	90.95	94.78	61.90
Pruning + beam 3	88,337	3.9	99.956%	90.95	94.78	61.90
Pruning, K=5 unk tags	77,735	3.5	99.961%	90.07	95.09	52.01
Pruning, K=20 unk tags	284,030	12.6	99.859%	91.24	95.29	60.49

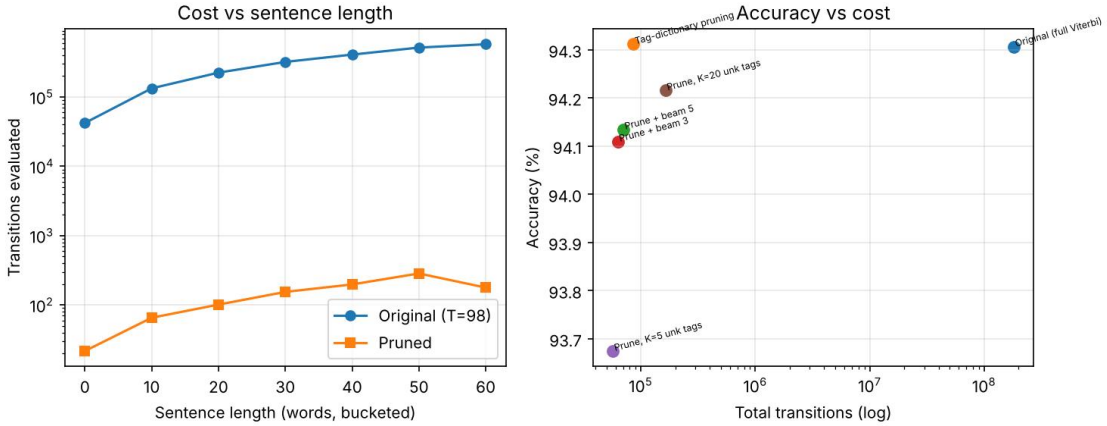


Figure 7.3 – Cost versus sentence length (left) and accuracy versus transitions (right)

Table 7.2 – The reduction grows with T

Tagset	T	Original transitions (news)	Pruned transitions (news)
Universal	12	2,728,236	78,353
Brown, suffixes removed	98	181,308,526	86,195

F. Defend the final design (2 marks)

Was the prediction supported? Yes for the main claim and in magnitude. Transitions fell from 181,308,526 to 86,195, a reduction of 99.952% (a factor of 2,103), against my objective of 95%. Per word the cost dropped from 9,160 to 4.4. Accuracy was identical on news (94.31% against 94.31%) and within 0.06 points on fiction (91.51% against 91.45%). The prediction about known words was right (known-word accuracy fell by 0.10 points, the paths through unseen word–tag pairs that the original sometimes got right) and unknown-word accuracy rose from 66.2% to 67.5%, because the candidate list rules out implausible tags. My estimate of about 100,000 transitions was close (actual 86,195). The beam predictions were partly wrong: beam 5 saved only 19% more transitions (against my 20% guess, which was near) but

cost 0.18 points of accuracy; beam 3 cost 0.2 points, so the beam is hardly worth its risk once pruning has been applied.

New costs and trade-offs: (1) a tag dictionary and signature table must be stored (a few tens of thousands of entries, small compared with the 181 million evaluations avoided); (2) the decoder can no longer output a tag never seen with a known word. This is the standard trade-off of a tag dictionary: for a domain with new usages of known words (a chat where "crash" becomes a verb) the pruned decoder would need the dictionary to be updated; (3) K = 5 loses almost 8 points on unknown words (59.8% against 67.5%), so K must not be too small, while K = 20 is slower and not more accurate than K = 10; (4) with T = 12 (universal tags) the saving is smaller in absolute terms (2.7 million to 78 thousand, still 35 times) because the original is already cheaper. When might the original still be preferable? When the tagset is tiny and the evaluation cost is irrelevant; when the training data is so small that the word–tag dictionary is unreliable and unseen word–tag pairs matter; and as the reference implementation for verifying the pruned decoder, since it is exact for the model. Final engineering conclusion: adopt tag-dictionary pruning with K = 10 and no beam; it reduces counted work by about three orders of magnitude at no measurable accuracy cost, and the cost now scales with the ambiguity of the text rather than with T^2.

Overall conclusion of the assignment

The experiments show one thread: counts decide models. Tokenisation decides how many events exist (Q1), the number of possible events grows exponentially in n (Q2), smoothing decides how scarce probability mass is shared (Q3), perplexity is meaningful only under fixed vocabularies (Q4), tagset granularity trades agreement against information (Q5), taggers differ in how they use local context and morphology (Q6), and exploiting sparsity of the word–tag relation removes almost all of Viterbi's computation (Q7).

Appendix A – Viva Preparation: Questions and Answers

Si.no	Question	Answer
1	Is the number of words in a text a fact?	No. It depends on the tokenisation policy. In Q1 the same Brown news text gave 88,997 tokens with whitespace splitting and 104,572 with my refined policy, and V fell from 19,067 to 11,897.
2	Why does an unsmoothed trigram give probability 0 to most sentences?	Any one unseen trigram makes the product zero; 82% of test trigrams were unseen and 918 of 925 test sentences contained one (Q2).
3	State the maximum-likelihood estimate for a bigram.	P(w_k | w_{k−1}) = C(w_{k−1} w_k) / C(w_{k−1}); it maximises training likelihood and gives zero to unseen pairs.
4	Why is add-one poor for n-grams?	It adds V pseudo-counts to every history and moves about 76–81% of the mass to unseen bigrams; seen counts are cut sharply (c* = 0.10 for a bigram seen 3 times after “)”).
5	How are the interpolation weights found?	By EM on held-out data: each held-out token is split among the components in proportion to λ_j p_j, and λ_j becomes the normalised total share; λ2 grew from 0.27 to 0.54 with data.
6	Why does backoff need the factor α(h)?	The discount D frees mass D × (seen types)/C(h) for each history; α(h) hands exactly this to unseen words, so the distribution sums to 1 (checked numerically).
7	Derive PP = 2^H.	PP = P(W)^(−1/N); take log2: log2 PP = −(1/N) log2 P(W) = H; so PP = 2^H.
8	Why can perplexities with different vocabularies not be compared?	They predict different events; hiding rare words in <UNK> lowered the bigram perplexity from 489 to 40 without improving the model (Table 4.3).
9	What does the shuffled-text result show?	Unigram perplexity is unchanged (336.7); bigram and trigram perplexities rise sharply, so the models capture local word order, not meaning.
10	What is the difference between open and closed classes?	Open: new members arise freely (nouns, verbs, adjectives, adverbs). Closed: small fixed sets of function words (determiners, prepositions, pronouns, conjunctions, modals).
11	How does tagset granularity affect agreement?	More tags mean more confusable boundaries: ambiguity rose from 6.9% (coarse) to 13.4% (Penn) of types, and kappa fell slightly (0.942 → 0.935).
12	Why is Viterbi O(n T^2)?	Each of n positions has T states, each maximising over T predecessors; transitions = (n−1)T^2 + T.
13	What does the Brill tagger learn?	An ordered list of transformations “change tag A to B if context”, chosen greedily by error reduction on training data; each is human readable.
14	What did your redesign change and why was it safe?	It restricted the candidate tags to those the lexicon allows (K = 10 tags for unknown words); this removed 99.95% of transitions with unchanged overall accuracy because impossible paths had probability 1e-8 and never won.
15	What limitation did the Q5 annotation have?	The second annotator was a statistical tagger, not a human re-annotation; kappa should be recomputed with my own re-annotation after a gap.

Appendix B – References

Bird, S., Klein, E. and Loper, E. (2009) Natural Language Processing with Python. O'Reilly. (NLTK)

Brill, E. (1995) Transformation-based error-driven learning and natural language processing. Computational Linguistics 21(4).

Francis, W. N. and Kučera, H. (1979) Brown Corpus Manual. Brown University.

Jelinek, F. and Mercer, R. L. (1980) Interpolated estimation of Markov source parameters from sparse data. Proc. Workshop on Pattern Recognition in Practice.

Jurafsky, D. and Martin, J. H. Speech and Language Processing, 3rd ed. draft: chapters on N-grams and Part-of-Speech Tagging.

Katz, S. M. (1987) Estimation of probabilities from sparse data for the language model component of a speech recogniser. IEEE Trans. ASSP 35(3).

Marcus, M., Santorini, B. and Marcinkiewicz, M. (1993) Building a large annotated corpus of English: the Penn Treebank. Computational Linguistics 19(2).

Ney, H., Essen, U. and Kneser, R. (1994) On structuring probabilistic dependences in stochastic language modelling. Computer Speech and Language 8.

Petrov, S., Das, D. and McDonald, R. (2012) A universal part-of-speech tagset. Proc. LREC.

Zipf, G. K. (1949) Human Behavior and the Principle of Least Effort. Addison-Wesley.

Appendix C – Source Code (fixed seed 42)

common.py

```
import os, re, random, math, json, collections
os.environ['NLTK_DATA']='/home/claude/nltk_data'
from nltk.corpus import brown
SEED=42
def sents(cat):
    return [[w.lower() for w in s] for s in brown.sents(categories=cat)]
def split(sents_, seed=SEED, frac=0.8):
    s=list(sents_); random.Random(seed).shuffle(s); k=int(len(s)*frac); return s[:k], s[k:]
def save(name,obj): json.dump(obj,open(f'res/{name}.json','w'),indent=1)
def detok(ws):
    t=' '.join(ws)
    t=re.sub(r' ([.,;:!?)%\]])',r'\1',t); t=re.sub(r'([(\[$]) ',r'\1',t); t=t.replace('`` ','"').replace(" "","")
    return t
```

lm.py

```
from common import *
def build_vocab(train,minc=2):
    c=collections.Counter(w for s in train for w in s)
    return {w for w,v in c.items() if v>=minc}
def mapunk(s,V): return [w if w in V else '<unk>' for w in s]
class Counts:
    def __init__(s,train,V,order=2):
        s.V=set(V)|{'<unk>','</s>'}; s.W=sorted(s.V); s.order=order
        s.c=[collections.Counter() for _ in range(order+1)]  # n-gram counts by length
        s.tr=[]
        for t in train:
            t=['<s>']*(order-1)+mapunk(t,V)+['</s>']; s.tr.append(t)
            for n in range(1,order+1):
                for i in range(order-1 if n>1 else order-1,len(t)-n+1):
                    pass
        # count n-grams ending at predicted positions
        for t in s.tr:
            for i in range(order-1,len(t)):
                for n in range(1,order+1): s.c[n][tuple(t[i-n+1:i+1])]+=1
        s.N=sum(s.c[1].values())
        s.hist=[collections.Counter() for _ in range(order+1)]  # context counts
        s.succ=[collections.defaultdict(set) for _ in range(order+1)]
        for n in range(2,order+1):
            for g,v in s.c[n].items(): s.hist[n][g[:-1]]+=v; s.succ[n][g[:-1]].add(g[-1])
    def prep(s,test):
```

```
        o=s.order; out=[]
        for t in test:
            t=['<s>']*(o-1)+mapunk(t,s.V-{'<unk>','</s>'} if False else {w for w in s.V})+['</s>']; out.append(t)
        return out
def uni(C,w): return (C.c[1][(w,)]+0.5)/(C.N+0.5*len(C.V))
# ---- bigram models
def p_addk(C,h,w,k):
    ch=C.hist[2].get((h,),0); return (C.c[2].get((h,w),0)+k)/(ch+k*len(C.V))
def make_backoff(C,D):
    unseen_mass_cache={}
    def p(h,w):
        ch=C.hist[2].get((h,),0)
        if ch==0: return uni(C,w)
        c=C.c[2].get((h,w),0)
        if c>0: return (c-D)/ch
        S=C.succ[2][(h,)]
        if h not in unseen_mass_cache: unseen_mass_cache[h]=sum(uni(C,x) for x in S)
        return D*len(S)/ch*uni(C,w)/(1-unseen_mass_cache[h])
    return p
class Interp:
    def __init__(s,C,order): s.C=C; s.o=order; s.lam=None
    def comps(s,ctx,w):
        C=s.C; o=s.o; p=[1/len(C.V)]; mask=[1]
        for n in range(1,o+1):
            h=tuple(ctx[len(ctx)-(n-1):]) if n>1 else ()
            if n==1: p.append(uni(C,w)); mask.append(1)
            else:
                ch=C.hist[n].get(h,0)
                if ch>0: p.append(C.c[n].get(h+(w,),0)/ch); mask.append(1)
                else: p.append(0.0); mask.append(0)
        return p,mask
    def fit(s,held,iters=30):
        data=[]
        for t in held:
            for i in range(s.o-1,len(t)): data.append(s.comps(t[:i],t[i]))
        k=s.o+1; lam=[1/k]*k
        for _ in range(iters):
            acc=[0.0]*k
            for p,m in data:
                z=[lam[j]*p[j]*m[j] for j in range(k)]; Z=sum(z)
                if Z>0:
                    for j in range(k): acc[j]+=z[j]/Z
            tot=sum(acc); lam=[a/tot for a in acc]
        s.lam=lam; return lam
    def p(s,ctx,w):
```

```
        pp,m=s.comps(ctx,w); z=sum(s.lam[j]*m[j] for j in range(len(pp)))
        return sum(s.lam[j]*pp[j]*m[j] for j in range(len(pp)))/z
def logp2(x): return math.log2(x)
```

taggers.py

```
from common import *
import numpy as np
SUF=['ing','ed','ly','ion','ness','ous','al','ble','ic','ive','er','est','s']
def sig(w):
    if any(c.isdigit() for c in w): return '<num>'
    if w[0].isupper(): return '<cap>'
    if '-' in w: return '<hyph>'
    for s in SUF:
        if w.endswith(s) and len(w)>len(s)+1: return '<-'+s+'>'
    return '<other>'
SIGS=['<num>','<cap>','<hyph>']+['<-'+s+'>' for s in SUF]+['<other>']
def tagged(cat,tagset=None,fine=False):
    S=brown.tagged_sents(categories=cat,tagset=tagset)
    out=[]
    for s in S:
        if fine: out.append([(w,re.sub(r'(-(TL|HL|NC)|\*|\+.*)$','',re.sub(r'(-(TL|HL|NC))+$','',t)).strip('*') or t) for w,t in s])
        else: out.append(list(s))
    return out
class HMM:
    def __init__(s,train,k=0.01):
        s.tags=sorted({t for x in train for _,t in x}); s.ti={t:i for i,t in enumerate(s.tags)}; T=len(s.tags); s.T=T
        trans=np.zeros((T+1,T)); em=collections.defaultdict(lambda: np.zeros(T)); hap=np.zeros((T,len(SIGS)))
        wc=collections.Counter(w.lower() for x in train for w,_ in x)
        for x in train:
            p=T
            for w,t in x:
                i=s.ti[t]; trans[p,i]+=1; em[w.lower()][i]+=1
                if wc[w.lower()]==1: hap[i,SIGS.index(sig(w))]+=1
                p=i
        tc=np.array([sum(em_[i] for em_ in [np.zeros(1)]) for i in range(0)]) if False else None
        tagc=np.zeros(T)
        for v in em.values(): tagc+=v
        s.logA=np.log((trans[:T]+k)/(trans[:T].sum(1,keepdims=True)+k*T));
s.logS=np.log((trans[T]+k)/(trans[T].sum()+k*T))
        s.em={w:v for w,v in em.items()}; s.tagc=tagc
        eps=1e-8
        s.logE={w:np.log(np.maximum(v/tagc,eps)) for w,v in s.em.items()}
        hs=hap.sum(1,keepdims=True)+0.1*len(SIGS)
```

```
        s.logSig=np.log((hap+0.1)/hs)  # P(sig|tag)
        s.sigtags={g:np.argsort(-hap[:,j])[:10] for j,g in enumerate(SIGS)}
        s.wordtags={w:np.nonzero(v)[0] for w,v in s.em.items()}
    def emit(s,w):
        l=w.lower()
        if l in s.logE: return s.logE[l],True
        return s.logSig[:,SIGS.index(sig(w))],False
    def viterbi(s,words,prune=False,beam=None):
        n=len(words); T=s.T; trans=0
        E=[s.emit(w) for w in words]
        def cand(i):
            if not prune: return np.arange(T)
            l=words[i].lower()
            if E[i][1]: return s.wordtags[l]
            return s.sigtags[sig(words[i])]
        c=cand(0); delta=s.logS[c]+E[0][0][c]; trans+=len(c); back=[]
        prev=c
        for i in range(1,n):
            if beam and len(prev)>beam:
                keep=np.argsort(-delta)[:beam]; prev=prev[keep]; delta=delta[keep]
            c=cand(i); M=delta[:,None]+s.logA[np.ix_(prev,c)]; trans+=len(prev)*len(c)
            b=M.argmax(0); delta=M[b,np.arange(len(c))]+E[i][0][c]; back.append((prev,b)); prev=c
        j=int(delta.argmax()); out=[prev[j]]
        for pv,b in reversed(back): j=int(b[j]); out.append(pv[j])
        trans+=0
        return [s.tags[i] for i in reversed(out)],trans
def acc(pred,gold,known=None):
    tot=ok=kt=kk=ut=uk=0
    for p,g,kn in zip(pred,gold,known):
        for a,b,k in zip(p,g,kn):
            tot+=1; ok+=a==b
            if k: kt+=1; kk+=a==b
            else: ut+=1; uk+=a==b
    return dict(overall=ok/tot,known=kk/max(kt,1),unknown=uk/max(ut,1),n=tot,nk=kt,nu=ut)
```

q1.py

```
from common import *
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
# --- own criteria tokenizer
EMO=r"[:;=][-']?[()DPp/]"
PAT=re.compile(rf"{EMO}|n't|'(?:s|m|re|ve|ll|d)\b|\d+(?:[.,]\d+)*(?![a-z0-9])|[a-z0-9]+(?:-[a-z0-9]+)*")
def own(t):
    t=t.lower(); t=re.sub(r"(\w)n't",r"\1 n't",t); return PAT.findall(t)
```

```
S=["Can't connect!! Net speed 4G/5G isn't consistent :(","Won't recharge via e-wallet","Rs.499 plan is value-for-money","It is the-best-network, no? ok ok","Super service da, semma speed"]
AMB="I'm gonna ship 10,000 units @ $5.99/ea. to St. John's; lol:-) U.S.A."
rows=[]
for s in S+[AMB]:
    tk=own(s); rows.append(dict(text=s,tokens=tk,N=len(tk),V=len(set(tk)),ws=len(s.split())))
# --- corpus policies
raw=brown.sents(categories='news'); text=' '.join(detok(s) for s in raw)
def p1(t): return t.split()
def p2(t): return re.findall(r"\w+|[^\w\s]",t)
def p3(t):
    t=t.lower(); t=re.sub(r"(\w)n't",r"\1 n't",t); t=re.sub(r"(\w)'(s|re|ve|ll|d|m)\b",r"\1 '\2",t)
    t=re.sub(r"\d+(?:[.,]\d+)*","<num>",t)
    return re.findall(r"<num>|n't|'\w+|[a-z]+(?:-[a-z]+)*|[^\w\s]",t)
out={}
fig,ax=plt.subplots(figsize=(6.5,4.5))
for name,f in [('P1 whitespace',p1),('P2 punctuation-aware',p2),('P3 refined',p3)]:
    tk=f(text); c=collections.Counter(tk); N=len(tk); V=len(c)
    fr=np.array(sorted(c.values(),reverse=True)); r=np.arange(1,len(fr)+1)
    sel=(r>=10)&(r<=2000); slope=np.polyfit(np.log10(r[sel]),np.log10(fr[sel]),1)[0]
    out[name]=dict(N=N,V=V,TTR=V/N,hapax=sum(1 for v in c.values() if
v==1),top10=c.most_common(10),slope=slope,hapax_pct=100*sum(1 for v in c.values() if v==1)/V)
    ax.loglog(r,fr,label=f'{name} (slope {slope:.2f})')
ax.set_xlabel('Rank (log)');ax.set_ylabel('Frequency (log)');ax.set_title('Rank-frequency, Brown
news');ax.legend();ax.grid(alpha=.3)
plt.tight_layout();plt.savefig('fig/q1_zipf.png',dpi=150)
save('q1',dict(strings=rows,policies=out,chars=len(text)))
for k,v in out.items(): print(k,{a:(round(b,3) if isinstance(b,float) else b) for a,b in v.items() if a!='top10'})
for r_ in rows: print(r_['N'],r_['V'],r_['tokens'])
```

q2.py

```
from common import *
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
rng=random.Random(SEED)
# synthetic helpdesk tickets (template based, seeded)
T=["my {d} is not {s}","please reset my {a} password","cannot connect to {n} from {p}","{d} is {s} since {t}","how do i install {sw} on my {d}","request access to {n} for {p}","the {n} is {s} again","need help with {sw} on {d}"]
F=dict(d=["laptop","printer","projector","phone","tablet"],s=["working","charging","starting","responding"],a=["wifi","email","portal","library"],n=["wifi","vpn","portal","server","lab"],p=["hostel","library","office","home"],t=["morning","yesterday","monday"],sw=["matlab","python","office","zoom"])
def ticket(): return rng.choice(T).format_map({k:rng.choice(v) for k,v in F.items()}).split()
tickets=[ticket() for _ in range(3000)]
news=sents('news'); fiction=sents('fiction')
```

```
edge=[rng.choice([["ok","thanks"],["reset","password"],["no","wifi"],["thanks","done"]]) for _ in range(3000)]
def grams(s,n):
    s=['<s>']*(n-1)+s+['</s>']; return [tuple(s[i:i+n]) for i in range(len(s)-n+1)]
def run(train,tests,name):
    res=[]
    for n in (1,2,3,4):
        C=collections.Counter(g for s in train for g in grams(s,n))
        for tname,test in tests.items():
            tg=[g for s in test for g in grams(s,n)]
            un=sum(1 for g in tg if g not in C)
            zs=sum(1 for s in test if any(g not in C for g in grams(s,n)))
            res.append(dict(dataset=name,test=tname,n=n,distinct=len(C),singleton=sum(1 for v in C.values() if
v==1),test_ngrams=len(tg),unseen_pct=100*un/len(tg),zero_sents=zs,test_sents=len(test)))
    return res
R=[]
tr,te=split(tickets); R+=run(tr,{'in-domain':te},'tickets')
tr,te=split(news); R+=run(tr,{'in-domain':te,'out-of-domain (fiction)':fiction[:len(te)]},'news')
tr,te=split(edge); R+=run(tr,{'in-domain':te},'edge')
# generation from MLE models trained on news
def gen(train,n,seed,maxlen=20):
    r=random.Random(seed); nxt=collections.defaultdict(collections.Counter)
    for s in train:
        g=['<s>']*(n-1)+s+['</s>']
        for i in range(len(g)-n+1): nxt[tuple(g[i:i+n-1])][g[i+n-1]]+=1
    h=tuple(['<s>']*(n-1)); out=[]
    for _ in range(maxlen):
        c=nxt[h]; w=r.choices(list(c),weights=list(c.values()))[0]
        if w=='</s>': break
        out.append(w); h=(h+(w,))[1:] if n>1 else ()
    return ' '.join(out)
trn,_=split(news); G={n:[gen(trn,n,s) for s in (1,2)] for n in (1,2,3,4)}
# overlap with training for 4-gram generated text
save('q2',dict(rows=R,gen=G))
for r in R: print(r['dataset'],r['test'],r['n'],r['distinct'],round(r['unseen_pct'],1),r['zero_sents'],r['test_sents'])
for n in G: print(n,G[n])
fig,ax=plt.subplots(figsize=(6.5,4))
for key,m in [('news in-domain','o-'),('news OOD','s--'),('tickets','^-'),('edge','d:')]:
    sel=[r for r in R if (key=='news in-domain' and r['dataset']=='news' and r['test']=='in-domain') or (key=='news OOD' and
r['test'].startswith('out')) or (key=='tickets' and r['dataset']=='tickets') or (key=='edge' and r['dataset']=='edge')]
    ax.plot([r['n'] for r in sel],[r['unseen_pct'] for r in sel],m,label=key)
ax.set_xticks([1,2,3,4]);ax.set_xlabel('n');ax.set_ylabel('% test n-grams
unseen');ax.legend();ax.grid(alpha=.3);plt.tight_layout();plt.savefig('fig/q2_unseen.png',dpi=150)
```

q3.py

```
from lm import *
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
news=sents('news'); fic=sents('fiction')
tr,rest=split(news); held=rest[:len(rest)//2]; test=rest[len(rest)//2:]
fic_t=fic[:len(test)]
rr=random.Random(SEED)
def noisy(ss,p=0.4): return [[('zq%d'%rr.randint(0,10**6) if rr.random()<p else w) for w in s] for s in ss]
edge=noisy(test)
conds={'matching (news)':test,'other domain (fiction)':fic_t,'edge (40% unknown)':edge}
res=[]; extra={}; Vfull=build_vocab(tr,2)
for frac in (0.1,0.5,1.0):
    sub=tr[:int(len(tr)*frac)]; V=Vfull; C=Counts(sub,V,2)
    I=Interp(C,2); H=[['<s>']+mapunk(t,V)+['</s>'] for t in held]; lam=I.fit(H)
    # tune k on held-out
    def ppl(pf,ss):
        tot=0;n=0
        for t in ss:
            t=['<s>']+mapunk(t,V)+['</s>']
            for i in range(1,len(t)): tot-=math.log2(pf(t[i-1],t[i])); n+=1
        return 2**(tot/n),n
    ks=[0.001,0.003,0.01,0.03,0.1,0.3,1.0]
    kbest=min(ks,key=lambda k:ppl(lambda h,w:p_addk(C,h,w,k),held)[0])
    n1=sum(1 for v in C.c[2].values() if v==1); n2=sum(1 for v in C.c[2].values() if v==2); D=n1/(n1+2*n2)
    bo=make_backoff(C,D)
    models={'add-one':lambda h,w:p_addk(C,h,w,1.0),f'add-k (k={kbest})':lambda h,w:p_addk(C,h,w,kbest),f'backoff
(D={D:.2f})':bo,'deleted interp.':lambda h,w:I.p([h],w)}
    for cn,ts in conds.items():
        for mn,pf in models.items():
            pp,n=ppl(pf,ts)
            # unseen mass averaged over test history tokens
            ms=[];
            for t in ts[:300]:
                t=['<s>']+mapunk(t,V)+['</s>']
                for i in range(1,len(t)):
                    h=t[i-1]; S=C.succ[2].get((h,),set())
                    ms.append(1-sum(pf(h,w) for w in S) if C.hist[2].get((h,),0)>0 else 1.0)

res.append(dict(frac=frac,cond=cn,model=mn,ppl=pp,unseen_mass=float(np.mean(ms)),train_tokens=C.N,V=len(C.V),lam=
lam,k=kbest,D=D))
    if frac==1.0:
        hs=[w for w,_ in sorted(((w,C.hist[2][(w,)]) for (w,) in [(x[0],) for x in C.hist[2]]),key=lambda x:-x[1])][:10] if False
else [h[0] for h,_ in C.hist[2].most_common(10)]
        chk=[]
```

```
    for h in hs:
      row={'h':h}
      for mn,pf in models.items(): row[mn]=sum(pf(h,w) for w in C.W)
      chk.append(row)
    # reconstituted counts
    seen=[g for g,v in C.c[2].items()]
    pick={}
    for g,v in C.c[2].most_common():
      if v in (1,2,3,5,10,20) and v not in pick and C.hist[2][(g[0],)]>=40: pick[v]=g
    cs=[]
    for v,g in sorted(pick.items()):
      ch=C.hist[2][(g[0],)]
      cs.append(dict(bigram='
'.join(g),c=v,ch=ch,addone=(v+1)*ch/(ch+len(C.V)),addk=(v+kbest)*ch/(ch+kbest*len(C.V)),backoff=bo(g[0],g[1])*ch,inte
rp=I.p([g[0]],g[1])*ch))
    extra=dict(check=chk,cstar=cs,predict_mass=float(np.mean([len(C.V)/(C.hist[2].get((t_[i-1],),1)+len(C.V)) for t in
test[:300] for t_ in [['<s>']+mapunk(t,V)+['</s>']] for i in range(1,len(t_))])))
save('q3',dict(res=res,extra=extra))
for r in res: print(r['frac'],r['cond'][:8],r['model'],round(r['ppl'],1),round(r['unseen_mass'],3),r['lam'] if r['model']=='deleted
interp.' and r['cond'].startswith('match') else '')
print(extra['check'][:3]); print(extra['cstar']); print(extra['predict_mass'])
fig,ax=plt.subplots(figsize=(6.5,4))
for mn in ['add-one','add-k','backoff','deleted']:
  sel=[r for r in res if r['cond'].startswith('match') and r['model'].startswith(mn)]
  ax.plot([r['frac']*100 for r in sel],[r['ppl'] for r in sel],'o-',label=mn)
ax.set_xlabel('% of training data');ax.set_ylabel('Held-out
perplexity');ax.set_yscale('log');ax.legend();ax.grid(alpha=.3);plt.tight_layout();plt.savefig('fig/q3_ppl.png',dpi=150)
```

q4.py

```
from lm import *
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
# toy distributions
def H(p): return -sum(x*math.log2(x) for x in p if x>0)
toys={'uniform-10':[0.1]*10,'skewed-10':[0.5,0.2,0.1,0.05,0.05,0.04,0.03,0.01,0.01,0.01],'dyadic-4':[0.5,0.25,0.125,0.125]}
toy={k:dict(H=H(p),PP=2**H(p)) for k,p in toys.items()}
q=toys['skewed-10']; u=toys['uniform-10']
toy['cross: true skewed, model uniform']=dict(H=-sum(a*math.log2(b) for a,b in zip(q,u)),PP=2**(-sum(a*math.log2(b) for
a,b in zip(q,u))))
toy['cross: true uniform, model skewed']=dict(H=-sum(a*math.log2(b) for a,b in zip(u,q)),PP=2**(-sum(a*math.log2(b) for
a,b in zip(u,q))))
news=sents('news'); fic=sents('fiction'); tr,rest=split(news); held=rest[:len(rest)//2]; test=rest[len(rest)//2:]
r=random.Random(SEED); shuf=[r.sample(s,len(s)) for s in test]
texts={'in-domain':test,'out-of-domain':fic[:len(test)],'shuffled':shuf}
```

```
out=[];
def evaluate(minc,order,skip_unk=False):
    V=build_vocab(tr,minc); C=Counts(tr,V,order); M=Interp(C,order)
    M.fit([['<s>']*(order-1)+mapunk(t,V)+['</s>'] for t in held],iters=20)
    res={}
    for tn,ts in texts.items():
        tot=0;n=0;unk=0;tokens=0
        for t in ts:
            t=['<s>']*(order-1)+mapunk(t,V)+['</s>']
            for i in range(order-1,len(t)):
                tokens+=1
                if t[i]=='<unk>': unk+=1
                if skip_unk and t[i]=='<unk>': continue
                tot-=math.log2(M.p(t[:i],t[i])); n+=1
        res[tn]=dict(H=tot/n,PP=2**(tot/n),unk=unk,tokens=tokens,V=len(C.V))
    return res,M.lam
for order,name in [(1,'unigram'),(2,'bigram'),(3,'trigram')]:
    res,lam=evaluate(2,order)
    for tn,v in res.items(): out.append(dict(model=name,text=tn,minc=2,skip=False,lam=lam,**v))
for minc in (1,2,5,10):
    for skip in (False,True):
        if minc==1 and skip: continue
        res,lam=evaluate(minc,2,skip)
        for tn,v in res.items(): out.append(dict(model='bigram',text=tn,minc=minc,skip=skip,lam=lam,**v))
save('q4',dict(toy=toy,rows=out))
for k,v in toy.items(): print(k,v)
for o in out: print(o['model'],o['text'],o['minc'],o['skip'],round(o['H'],2),round(o['PP'],1),o['unk'],o['tokens'],o['V'])
```

q5.py

```
from taggers import *
from nltk.corpus import treebank
import numpy as np
S=[[(w,t) for w,t in s if t!='-NONE-'] for s in treebank.tagged_sents()]
U=[[u for u,p in zip(su,sp) if p[1]!='-NONE-'] for su,sp in
zip(treebank.tagged_sents(tagset='universal'),treebank.tagged_sents())]
CMPL=['fail','crash','drop','lag','hang','freeze','disconnect','break','broke','stop','refus','complain','delay','cancel','block','slow','r
eject','los','lost','cut']
def custom(w,t):
    l=w.lower()
    if t.startswith('VB') and any(l.startswith(c) for c in CMPL): return 'V_CMPL'
    if t.startswith('VB'): return 'V_GEN'

m={'NN':'NOUN','NNS':'NOUN','NNP':'PROD','NNPS':'PROD','MD':'MODAL','JJ':'ADJ','JJR':'ADJ','JJS':'ADJ','RB':'ADV',
```

```
'RBR':'ADV','RBS':'ADV','CD':'NUM','PRP':'PRON','PRP$':'PRON','DT':'DET','IN':'PREP','CC':'CONJ','TO':'TO','UH':'INTJ
','SYM':'SYM','FW':'TAMIL_MIX','EX':'DET','WDT':'WH','WP':'WH','WP$':'WH','WRB':'WH','RP':'PART','POS':'PART','P
DT':'DET','LS':'SYM','$':'SYM','#':'SYM'}
    if t in m:
        r=m[t]
        if r=='PROD' and w.isupper() and len(w)<=4: return 'ABBR'
        return r
    if t in ('.',',',':','``','"','-LRB-','-RRB-'): return 'PUNCT'
    return 'OTHER'
MAP={'Coarse (Universal, 12)':lambda w,t,u:u,'Penn Treebank (46)':lambda w,t,u:t,'Custom domain tagset':lambda
w,t,u:custom(w,t)}
rnd=random.Random(SEED); idx=list(range(len(S))); rnd.shuffle(idx)
ann=idx[:100]; rest=idx[100:]; k=int(len(rest)*0.8); trn=rest[:k]; tst=rest[k:]
def conv(ids,f): return [[(S[i][j][0],f(S[i][j][0],S[i][j][1],U[i][j][1])) for j in range(len(S[i]))] for i in ids]
def kappa(a,b):
    n=len(a); labs=sorted(set(a)|set(b)); ia={l:i for i,l in enumerate(labs)}; M=np.zeros((len(labs),len(labs)))
    for x,y in zip(a,b): M[ia[x],ia[y]]+=1
    po=np.trace(M)/n; pe=(M.sum(0)*M.sum(1)).sum()/n**2; return (po-pe)/(1-pe)
out=[]
for name,f in MAP.items():
    allc=conv(range(len(S)),f)
    types=collections.defaultdict(set)
    for s in allc:
        for w,t in s: types[w.lower()].add(t)
    amb=100*sum(1 for v in types.values() if len(v)>1)/len(types); avg=np.mean([len(v) for v in types.values()])
    tagset=sorted({t for s in allc for _,t in s})
    H=HMM(conv(trn+ann[:0],f))  # trained on disjoint sentences
    # annotator B proxy: HMM trained on disjoint sentences tags the 100 annotation sentences
    A=conv(ann,f); gold=[[t for _,t in s] for s in A]; pred=[H.viterbi([w for w,_ in s])[0] for s in A]
    a=[t for g in gold for t in g]; b=[t for p in pred for t in p]
    cf=collections.Counter((x,y) for x,y in zip(a,b) if x!=y)
    # tagger accuracy on held-out
    T_=conv(tst,f); gt=[[t for _,t in s] for s in T_]; kn=[[w.lower() in H.logE for w,_ in s] for s in T_]
    pt=[H.viterbi([w for w,_ in s])[0] for s in T_]; ac=acc(pt,gt,kn)
    out.append(dict(tagset=name,n_tags=len(tagset),kappa=kappa(a,b),agree=sum(x==y for x,y in
zip(a,b))/len(a),amb_pct=amb,avg_tags=avg,confusions=[(x,y,c) for (x,y),c in
cf.most_common(5)],hmm_acc=ac['overall'],hmm_known=ac['known'],hmm_unk=ac['unknown'],tokens_annot=len(a)))
    if name.startswith('Custom'): out[-1]['tags']=tagset
save('q5',out)
for o in out:
print(o['tagset'],o['n_tags'],round(o['kappa'],4),round(o['agree'],4),round(o['amb_pct'],2),round(o['avg_tags'],3),round(o['hmm
_acc'],4),o['confusions'])
print(out[-1]['tags'])
```

q6.py

```
from taggers import *
import time
news=tagged('news','universal'); fic=tagged('fiction','universal')[:1500]
tr,te=split(news)
rtest=random.Random(SEED);
def mft(train):
    c=collections.defaultdict(collections.Counter)
    for x in train:
        for w,t in x: c[w.lower()][t]+=1
    return c
PUNCT=set(list('.,;:!?`\'"()-')+['``',"''",'--'])
def rule_tagger(lex,words,stats):
    tags=[]; base=[]
    for i,w in enumerate(words):
        l=w.lower(); base.append(lex[l].most_common(1)[0][0] if l in lex else None)
    tags=[b if b else 'NOUN' for b in base]
    def poss(l): return set(lex[l]) if l in lex else set()
    for i,w in enumerate(words):
        l=w.lower(); unk=base[i] is None; prev=tags[i-1] if i else '<s>'; prevw=words[i-1].lower() if i else ''
        def fire(r,t): stats[r]=stats.get(r,0)+1; tags[i]=t
        if w in PUNCT or all(not c.isalnum() for c in w): fire('R1 punct->','.'); continue
        if unk and any(c.isdigit() for c in w): fire('R2 digit->NUM','NUM')
        elif unk and w[0].isupper() and i>0: fire('R3 capitalised->NOUN','NOUN')
        elif unk and l.endswith('ing'): fire('R4 -ing->VERB','VERB')
        elif unk and l.endswith('ed'): fire('R5 -ed->VERB','VERB')
        elif unk and l.endswith('ly'): fire('R6 -ly->ADV','ADV')
        elif unk and re.search(r'(ous|ful|ive|al|able|ible|ic|less)$',l): fire('R7 adj suffix->ADJ','ADJ')
        elif unk and l.endswith('s') and prev in ('DET','ADJ'): fire('R8 DET+..s->NOUN','NOUN')
        elif prevw=='to' and 'VERB' in poss(l) and tags[i]!='VERB': fire('R9 to+X->VERB','VERB')
        elif prev=='DET' and tags[i]=='VERB' and 'NOUN' in poss(l): fire('R10 DET+VERB?->NOUN','NOUN')
        elif prev in('ADJ',) and tags[i]=='VERB' and 'NOUN' in poss(l): fire('R11 ADJ+VERB?->NOUN','NOUN')
        elif prev=='PRON' and tags[i]=='NOUN' and 'VERB' in poss(l): fire('R12 PRON+NOUN?->VERB','VERB')
        elif prev=='ADP' and tags[i]=='VERB' and 'NOUN' in poss(l) and l.endswith('ing')==False: fire('R13 ADP+VERB?-
>NOUN','NOUN')
    return tags
def run(train,tests,frac):
    lex=mft(train); vocab=set(lex); H=HMM(train); out=[]
    for tn,ts in tests.items():
        gold=[[t for _,t in x] for x in ts]; known=[[w.lower() in vocab for w,_ in x] for x in ts]
        pb=[[lex[w.lower()].most_common(1)[0][0] if w.lower() in vocab else 'NOUN' for w,_ in x] for x in ts]
        st={}; pr=[rule_tagger(lex,[w for w,_ in x],st) for x in ts]
        trs=0; ph=[]
        for x in ts:
```

```
            p,t=H.viterbi([w for w,_ in x]); ph.append(p); trs+=t
        for nm,p,extra in [('MFT baseline',pb,{}),('Rule-based',pr,dict(rule_fires=st)),('HMM+Viterbi',ph,dict(transitions=trs))]:
            r=acc(p,gold,known); r.update(model=nm,test=tn,frac=frac,**extra); out.append(r)
        # confusion
        if frac==1.0:
            for nm,p in [('MFT',pb),('Rule',pr),('HMM',ph)]:
                cf=collections.Counter((g,q) for gg,pp in zip(gold,p) for g,q in zip(gg,pp) if g!=q)
                out.append(dict(confusion=nm,test=tn,top=[(a,b,c) for (a,b),c in cf.most_common(8)]))
            if tn=='news':
                import pickle; pickle.dump((gold,known,pb,pr,ph),open('res/q6_preds.pkl','wb'))
    return out
R=[]
for frac in (0.1,0.5,1.0):
    sub=tr[:int(len(tr)*frac)]
    R+=run(sub,{'news':te,'out-of-domain':fic},frac)
    print('frac',frac,flush=True)
save('q6',R)
for r in R:
    if 'model' in r:
print(r['frac'],r['test'][:4],r['model'],round(r['overall']*100,2),round(r['known']*100,2),round(r['unknown']*100,2),r.get('transit
ions',''))
    else: print(r['confusion'],r['test'],r['top'][:5])
print(sum(len(x) for x in te),len(te))
```

q6_brill.py

```
import sys,time
from taggers import *
import nltk
from nltk.tag import brill, brill_trainer
from nltk.tbl import Feature
frac=float(sys.argv[1])
news=tagged('news','universal'); fic=tagged('fiction','universal')[:1500]
tr,te=split(news); sub=tr[:int(len(tr)*frac)]
cut=int(len(sub)*0.6); A,B=sub[:cut],sub[cut:]
lex=collections.defaultdict(collections.Counter)
for x in A:
    for w,t in x: lex[w.lower()][t]+=1
class Lex(nltk.tag.TaggerI):
    def tag(s,ws): return [(w,lex[w.lower()].most_common(1)[0][0] if w.lower() in lex else 'NOUN') for w in ws]
class Suf(Feature):
    json_tag='Suf'
    @staticmethod
    def extract_property(tokens,i): return tokens[i][0][-3:]
```

```
class Cap(Feature):
   json_tag='Cap'
   @staticmethod
   def extract_property(tokens,i): return tokens[i][0][:1].isupper()
base=Lex()
P,W=brill.Pos,brill.Word
T=[brill.Template(P([-1])),brill.Template(P([1])),brill.Template(P([-2,-1])),brill.Template(W([-
1])),brill.Template(W([0]),P([-1])),brill.Template(P([-1]),P([1])),brill.Template(Suf([0])),brill.Template(Suf([0]),P([-
1])),brill.Template(Cap([0])),brill.Template(Cap([0]),P([-1])),brill.Template(W([1]))]
t0=time.time()
tr_=brill_trainer.BrillTaggerTrainer(base,T,trace=0)
tg=tr_.train(B,max_rules=40,min_score=3)
vocab={w.lower() for x in sub for w,_ in x}
def ev(tagger,ts):
   gold=[[t for _,t in x] for x in ts]; known=[[w.lower() in vocab for w,_ in x] for x in ts]
   pred=[[t for _,t in tagger.tag([w for w,_ in x])] for x in ts]
   return acc(pred,gold,known)
rules=tg.rules(); st=tg.train_stats()
out=dict(frac=frac,n_rules=len(rules),rulescores=st['rulescores'],initialerrors=st['initialerrors'],finalerrors=st['finalerrors'],trai
n_secs=time.time()-t0,rules=[str(r) for r in rules[:12]],curve=[],nA=sum(map(len,A)),nB=sum(map(len,B)))
for k in sorted({0,1,2,3,5,8,12,20,30,len(rules)}):
   if k>len(rules): continue
   tk=brill.BrillTagger(base,rules[:k])
   out['curve'].append(dict(k=k,news=ev(tk,te),ood=ev(tk,fic)))
save(f'q6_brill_{int(frac*100)}',out)
print(frac,len(rules),round(out['train_secs'],1),[(c['k'],round(c['news']['overall']*100,2),round(c['news']['unknown']*100,1))
for c in out['curve']])
print(out['rules'][:5])
```

q7.py

```
from taggers import *
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
news=tagged('news',None,fine=True); fic=tagged('fiction',None,fine=True)[:1500]
tr,te=split(news); H=HMM(tr); vocab=set(H.em); T=H.T
print('T',T,len(vocab))
def runcfg(ts,prune,beam=None,K=None):
   if K:
      H.sigtags={g:np.argsort(-H.logSig[:,j])[:K] for j,g in enumerate(SIGS)}
   preds=[];tot=0;lens=[]
   for x in ts:
      p,t=H.viterbi([w for w,_ in x],prune=prune,beam=beam); preds.append(p); tot+=t; lens.append((len(x),t))
   gold=[[t for _,t in x] for x in ts]; kn=[[w.lower() in vocab for w,_ in x] for x in ts]
   a=acc(preds,gold,kn); a.update(transitions=tot,per_word=tot/a['n'],lens=lens); return a
```

```
R=[]
cfgs=[('Original (full Viterbi)',False,None,None),('Tag-dictionary pruning',True,None,10),('Pruning + beam 5',True,5,10),('Pruning + beam 3',True,3,10),('Pruning, K=5 unk tags',True,None,5),('Pruning, K=20 unk tags',True,None,20)]
for tn,ts in [('news',te),('out-of-domain',fic)]:
    for nm,pr,bm,K in cfgs:
        r=runcfg(ts,pr,bm,K); r.update(cfg=nm,test=tn); R.append(r);
print(tn,nm,round(r['overall']*100,2),round(r['known']*100,2),round(r['unknown']*100,2),r['transitions'],round(r['per_word'],1),flush=True)
# tagset-size scaling (original vs pruned) using universal tags
nu=tagged('news','universal'); tru,teu=split(nu); Hu=HMM(tru);
tr_u=sum(Hu.viterbi([w for w,_ in x])[1] for x in teu); pr_u=sum(Hu.viterbi([w for w,_ in x],prune=True)[1] for x in teu)
H.sigtags={g:np.argsort(-H.logSig[:,j])[:10] for j,g in enumerate(SIGS)}
# sentence length vs transitions
import collections as C
def by_len(r):
    d=C.defaultdict(list)
    for L,t in r['lens']: d[(L//10)*10].append(t)
    return {k:float(np.mean(v)) for k,v in sorted(d.items())}
o=[r for r in R if r['test']=='news' and r['cfg'].startswith('Original')][0]; p=[r for r in R if r['test']=='news' and r['cfg']=='Tag-dictionary pruning'][0]
bl_o=by_len(o); bl_p=by_len(p)
save('q7',dict(R=[{k:v for k,v in r.items() if k!='lens'} for r in R],T=T,universal=dict(T=Hu.T,orig=tr_u,pruned=pr_u,n=sum(len(x) for x in teu)),bylen=dict(orig=bl_o,pruned=bl_p)))
fig,ax=plt.subplots(1,2,figsize=(10,4))
ax[0].plot(list(bl_o),list(bl_o.values()),'o-',label=f'Original (T={T})');ax[0].plot(list(bl_p),list(bl_p.values()),'s-',label='Pruned');ax[0].set_yscale('log');ax[0].set_xlabel('Sentence length (words, bucketed)');ax[0].set_ylabel('Transitions evaluated');ax[0].legend();ax[0].grid(alpha=.3);ax[0].set_title('Cost vs sentence length')
news_r=[r for r in R if r['test']=='news']
for r in news_r: ax[1].scatter(r['transitions'],r['overall']*100,s=50);
ax[1].annotate(r['cfg'].replace('Pruning','Prune'),(r['transitions'],r['overall']*100),fontsize=6,rotation=15)
ax[1].set_xscale('log');ax[1].set_xlabel('Total transitions (log)');ax[1].set_ylabel('Accuracy (%)');ax[1].grid(alpha=.3);ax[1].set_title('Accuracy vs cost');plt.tight_layout();plt.savefig('fig/q7_cost.png',dpi=150)
print(bl_o,bl_p,tr_u,pr_u)
```

figs.py

```
import json,glob,matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt, numpy as np
from matplotlib.patches import FancyBboxPatch, Polygon
plt.rcParams['font.family']='serif'; plt.rcParams['font.serif']=['Times New Roman','Liberation Serif','DejaVu Serif']
L=lambda n: json.load(open(f'res/{n}.json'))
def flow(rows,fname,title=None,decision=()):
    """rows: list of rows; each row list of box texts; arrows between consecutive boxes in reading order (snake)."""
```

```
    nr=len(rows); nc=max(len(r) for r in rows); fig,ax=plt.subplots(figsize=(2.6*nc+0.6,1.35*nr+0.5)); ax.set_xlim(0,nc); ax.set_ylim(0,nr); ax.axis('off')
    pos=[]
    for ri,r in enumerate(rows):
        y=nr-ri-0.5; xs=[(i+0.5)*nc/len(r) for i in range(len(r))]
        for x,t in zip(xs,r):
            w=0.88*nc/len(r)*(1 if len(r)>1 else 0.6); h=0.62
            if t in decision:
                ax.add_patch(Polygon([[x-w/2,y],[x,y+h/2+.05],[x+w/2,y],[x,y-h/2-.05]],fc='#fff2cc',ec='black'))
            else:
                ax.add_patch(FancyBboxPatch((x-w/2,y-h/2),w,h,boxstyle='round,pad=0.02,rounding_size=0.06',fc='#dae8fc',ec='black'))
            ax.text(x,y,t,ha='center',va='center',fontsize=8.5,wrap=True); pos.append((x,y,w,h))
    for a,b in zip(pos[:-1],pos[1:]):
        if abs(a[1]-b[1])<1e-6: ax.annotate('',xy=(b[0]-b[2]/2,b[1]),xytext=(a[0]+a[2]/2,a[1]),arrowprops=dict(arrowstyle='->'))
        else: ax.annotate('',xy=(b[0],b[1]+b[3]/2+.03),xytext=(a[0],a[1]-a[3]/2-.03),arrowprops=dict(arrowstyle='->',connectionstyle='arc3,rad=0.0'))
    if title: ax.set_title(title,fontsize=10)
    plt.tight_layout(); plt.savefig(f'fig/{fname}.png',dpi=170); plt.close()
flow([['Raw review\nstring','Lowercase\n(case folding)','Split clitics\n(n\'t, \'s, \'m)'],['Keep emoticons\nas one token','Keep hyphen\ncompounds','Numbers /\ncurrency split'],['Drop bare\npunctuation','Token list','Count N, V,\nhapax']],'flow_q1','Own tokenisation pipeline (criteria C1-C6)')
flow([['Corpus\n(Brown news)','Sentence split\n80/20, seed 42','Pad <s> ... </s>'],['Count n-grams\n(n=1..4)','Test n-gram\nin train table?','Unseen % /\nzero-prob sentences'],['Generate text\nby sampling','Compare n','Choose n']],'flow_q2','Unsmoothed n-gram experiment',decision=('Test n-gram\nin train table?',))
flow([['Train (10/50/100%)','Fixed vocab,\nmin-count 2 -> <UNK>','Fit add-one /\nadd-k'],['Backoff with\ndiscount D','Deleted\ninterpolation\n(EM on held-out)','Check sum = 1\n(10 histories)'],['Perplexity on\n3 test sets','Mass on unseen\nbigrams','Report c*, lambda']],'flow_q3','Smoothing experiment pipeline')
flow([['Test text','Map OOV to <UNK>','Model P(w|history)'],['Sum -log2 P','Divide by N\n= cross-entropy H','Perplexity = 2^H']],'flow_q4','From probabilities to perplexity')
flow([['Treebank sentences\n(3,914)','Map tags:\nUniversal / PTB /\nCustom','Annotate 100\nsentences twice'],['Cohen kappa','Ambiguity:\n% types, tags/type','Top-5 confused\npairs'],['Tagger accuracy\nper tagset','Application:\nproducts + complaint\nverbs','Choose tagset']],'flow_q5','Tagset evaluation workflow')
flow([['Tagged Brown\nnews (universal)','Split 80/20\nseed 42','Train 10/50/100%'],['MFT\nbaseline','Rule-based\n(13 rules)','HMM + Viterbi\n(signature UNK)'],['Brill: learn\nordered rules\nfrom baseline','Evaluate: all /\nknown / unknown','Compare in-domain\nvs out-of-domain']],'flow_q6','Three tagging approaches')
flow([['Diagnose:\nT^2 per word','Objective:\n>=95% fewer\ntransitions','Prune to words\' seen tags'],['Unknown words:\ntop-K tags by\nsuffix signature','Optional beam B','Validate:\ncounts + accuracy'],['Prediction\nsupported?','Trade-offs','Final design']],'flow_q7','Q7 redesign data-flow')
# trellis
fig,ax=plt.subplots(1,2,figsize=(9,3.6)); T=6; n=4
for a,mode in zip(ax,['Original: full trellis','Redesign: pruned trellis']):
    a.set_title(mode,fontsize=10); a.axis('off')
    cand=[list(range(T))]*n if mode.startswith('Orig') else [[1,3],[0,4],[2],[1,5]]
    for i in range(n):
```

```
        for s in range(T): a.plot(i,s,'o',color='#888' if s not in cand[i] else '#1f77b4',ms=9 if s in cand[i] else 6)
    for i in range(n-1):
        for p in cand[i]:
            for c in cand[i+1]: a.plot([i,i+1],[p,c],color='#1f77b4',lw=0.5,alpha=.6)
    for i,w in enumerate(['they','can','fish','.']): a.text(i,-1,w,ha='center')
    a.text(1.5,T,'transitions: '+(f'{(n-1)*T*T}' if mode.startswith('Orig') else f'{sum(len(cand[i])*len(cand[i+1]) for i in
range(n-1))}'),ha='center')
plt.tight_layout(); plt.savefig('fig/trellis.png',dpi=170); plt.close()
# charts
q4=L('q4'); rows=q4['rows']
fig,ax=plt.subplots(figsize=(6.5,4)); w=0.25
for j,tx in enumerate(['in-domain','out-of-domain','shuffled']):
    v=[next(r['PP'] for r in rows if r['model']==m and r['text']==tx and r['minc']==2 and not r['skip']) for m in
('unigram','bigram','trigram')]
    ax.bar(np.arange(3)+j*w,v,w,label=tx)
ax.set_xticks(np.arange(3)+w);ax.set_xticklabels(['unigram','bigram','trigram']);ax.set_ylabel('Perplexity');ax.legend();ax.gri
d(axis='y',alpha=.3);plt.tight_layout();plt.savefig('fig/q4_pp.png',dpi=150);plt.close()
q6=L('q6'); fig,ax=plt.subplots(1,2,figsize=(10,4))
for k,tn in enumerate(['news','out-of-domain']):
    for nm in ['MFT baseline','Rule-based','HMM+Viterbi']:
        s=[r for r in q6 if r.get('model')==nm and r['test']==tn]; ax[k].plot([r['frac']*100 for r in s],[r['overall']*100 for r in s],'o-
',label=nm)
    b=[json.load(open(f'res/q6_brill_{p}.json')) for p in (10,50,100)]
    ax[k].plot([10,50,100],[x['curve'][-1]['news' if k==0 else 'ood']['overall']*100 for x in b],'s--',label='Brill')
    ax[k].set_title(tn);ax[k].set_xlabel('% training data');ax[k].set_ylabel('Accuracy
(%)');ax[k].grid(alpha=.3);ax[k].legend(fontsize=8)
plt.tight_layout();plt.savefig('fig/q6_acc.png',dpi=150);plt.close()
b=json.load(open('res/q6_brill_100.json')); fig,ax=plt.subplots(figsize=(6.5,4))
ax.plot([c['k'] for c in b['curve']],[c['news']['overall']*100 for c in b['curve']],'o-',label='news');ax.plot([c['k'] for c in
b['curve']],[c['ood']['overall']*100 for c in b['curve']],'s-',label='out-of-domain');ax.set_xlabel('Transformations
applied');ax.set_ylabel('Test accuracy
(%)');ax.legend();ax.grid(alpha=.3);plt.tight_layout();plt.savefig('fig/q6_brill.png',dpi=150);plt.close()
q5=L('q5'); fig,ax=plt.subplots(figsize=(6.5,4)); x=np.arange(3)
ax.bar(x-0.2,[o['kappa'] for o in q5],0.4,label="Cohen's kappa");ax.bar(x+0.2,[o['amb_pct']/100 for o in
q5],0.4,label='ambiguous types (fraction)')
ax.set_xticks(x);ax.set_xticklabels(['Coarse','PTB','Custom']);ax.legend();ax.grid(axis='y',alpha=.3);plt.tight_layout();plt.sav
efig('fig/q5_bars.png',dpi=150);plt.close()
```

run_all.py

```
# Re-runs every experiment (seed 42). Requires nltk data in ./nltk_data (brown, treebank, universal_tagset).
import subprocess,sys
for f in ['q1','q2','q3','q4','q5','q6']: subprocess.run([sys.executable,f+'.py'],check=True)
for fr in ['0.1','0.5','1.0']: subprocess.run([sys.executable,'q6_brill.py',fr],check=True)
subprocess.run([sys.executable,'q7.py'],check=True); subprocess.run([sys.executable,'figs.py'],check=True)
```